Università degli Studi di Padova

Dipartimento di Matematica "Tullio Levi-Civita"

Corso di Laurea in Informatica



Progettazione e sviluppo di un Relying Party conforme allo standard EUDI Wallet: dal parsing di query DCQL alla convalida crittografica

Tesi di laurea triennale

Relatore

Prof. Giovanni Da San Martino

Laureando

Andrea Masiero

Anno Accademico 2025-2026

Sommario

Il presente documento descrive l'attività svolta dal laureando Andrea Masiero durante il periodo di stage presso l'azienda Monokee. L'esperienza, svolta a conclusione del corso di laurea triennale in Informatica, ha avuto una durata complessiva di trecentoventi ore.

Il lavoro di tesi illustra la progettazione e lo sviluppo di un *Relying Party* in ambiente Node.js/TypeScript, conforme alle specifiche dell'ecosistema EUDI Wallet, con particolare attenzione al protocollo OpenID for Verifiable Presentations (OpenID4VP). Il progetto copre l'intero ciclo di vita dell'interazione con il Wallet, focalizzandosi in particolare su due aspetti tecnici avanzati: le richieste dinamiche e la validazione crittografica. Da un lato, il sistema implementa il parsing e l'elaborazione del Dynamic Credential Query Language (DCQL), permettendo al *Relying Party* di formulare richieste complesse, flessibili e multi-credenziale, superando i limiti delle query statiche. Dall'altro, il nucleo del progetto si concentra sulla rigorosa verifica delle credenziali, supportando nativamente il formato SD-JWT (Selective Disclosure JWT). Il risultato finale è un componente architetturale robusto, modulare e pronto per essere integrato in scenari reali di identità digitale, dimostrando la fattibilità tecnica e l'efficacia dei nuovi standard europei per lo scambio sicuro di credenziali verificabili.

Indice

Glossary ix

Acronyms xii

1 Il contesto aziendale 1
- 1.1 Scelta dello stage . . . 1
- 1.2 Dominio applicativo . . . 1
- 1.3 Tecnologie utilizzate . . . 2
  - 1.3.1 TypeScript . . . 2
  - 1.3.2 Node.js . . . 2
  - 1.3.3 Express.js . . . 2
  - 1.3.4 MongoDB . . . 2
  - 1.3.5 Redis . . . 3
  - 1.3.6 jose (JavaScript Object Signing and Encryption) . . . 3
  - 1.3.7 Docker . . . 3
  - 1.3.8 ESLint . . . 3
  - 1.3.9 Prettier . . . 3
  - 1.3.10 Yarn . . . 3
  - 1.3.11 Nodemon . . . 4
  - 1.3.12 dotenv . . . 4
  - 1.3.13 Librerie Interne Monokee . . . 4
  - 1.3.14 Azure Container Registry (ACR) . . . 4

2 Analisi dei Requisiti 7
- 2.1 Panoramica e Attori . . . 7
  - 2.1.1 Attori del sistema . . . 7
- 2.2 Casi d'uso . . . 8
- 2.3 Tracciamento dei requisiti . . . 38
  - 2.3.1 Requisiti funzionali . . . 39
  - 2.3.2 Requisiti qualitativi . . . 40
  - 2.3.3 Requisiti di vincolo . . . 41
  - 2.3.4 Riepilogo quantitativo . . . 41

3 Progettazione 43
- 3.1 Architettura generale . . . 43
  - 3.1.1 Stratificazione architetturale . . . 44
    - 3.1.1.1 Presentation Layer (Presentazione e instradamento) . . . 44
    - 3.1.1.2 Business Logic Layer (Logica applicativa e servizi) . . . 45
    - 3.1.1.3 Data Model Layer (Modelli di dominio e contratti dati) . . . 45
    - 3.1.1.4 Data Access Layer (Accesso ai dati e persistenza) . . . 45
- 3.2 Organizzazione della codebase . . . 46
  - 3.2.1 Criteri di modularità e convenzioni . . . 46

3.2.2 Decomposizione logica dei moduli per layer . . . 47
3.2.2.1 Layer dei modelli di dominio (`src/models/`) . . . 47
3.2.2.2 Layer di presentazione e instradamento (`src/routes/`) . . . 47
3.2.2.3 Layer di logica applicativa (`src/services/`) . . . 47
3.2.2.4 Layer di accesso ai dati (`src/repositories/`) . . . 48
3.3 Modellazione dei dati di dominio . . . 48
3.3.1 Entità persistenti: il modello ClientModel e le politiche di sicurezza 48
3.3.2 Strutture transazionali ed effimere: query DCQL e VP Token_{G} . . . 48
3.3.3 Contesti di validazione e Data Transfer Object (DTO) . . . 49
3.4 Strategia di persistenza e Data Access Layer . . . 49
3.4.1 Storage volatile ad alte prestazioni: Redis . . . 49
3.4.2 Storage persistente e multi-tenant: MongoDB . . . 50
3.5 Sicurezza crittografica e gestione della fiducia . . . 50
3.5.1 Forward secrecy applicativa mediante JWE e chiavi effimere . . . 50
3.5.2 Risoluzione gerarchica della fiducia: OpenID Federation 1.0 e Trust Chains . . . 50
3.5.3 Validazione crittografica di SD-JWT e prevenzione dei Replay Attack 51

4 Codifica e implementazione **53**
4.1 Metodologia e organizzazione del lavoro . . . 53
4.1.1 Studio del contesto e degli standard . . . 53
4.1.2 Approccio incrementale *bottom-up* . . . 53
4.2 Flussi operativi del protocollo OID4VP . . . 54
4.2.1 Inizializzazione della sessione . . . 54
4.2.2 Generazione del Request Object . . . 55
4.2.3 Elaborazione della risposta del Wallet . . . 56
4.2.4 Ritiro dei dati verificati . . . 56
4.2.5 Validazione crittografica SD-JWT e DCQL . . . 56
4.2.6 Risoluzione delle Trust Chain . . . 57
4.3 Sfide tecniche e soluzioni adottate . . . 57
4.3.1 Atomicità del consumo di sessione in un sistema distribuito . . . 57
4.3.2 Forward Secrecy e distruzione sicura delle chiavi private . . . 57
4.3.3 Verifica della corrispondenza chiave privata/certificato X.509 . . . 58
4.3.4 Workaround per hostname Docker in ambiente di sviluppo . . . 58
4.3.5 Conversione bidirezionale *camelCase–snake_case* con eccezioni . . . 59
4.3.6 Discriminazione tra payload in chiaro e payload cifrato . . . 59

5 Verifica e validazione **61**
5.1 Verifica statica . . . 61
5.1.1 Type checking e compilazione TypeScript . . . 61
5.1.2 Analisi statica con ESLint . . . 62
5.1.3 Audit automatico delle dipendenze . . . 62
5.1.4 Misure di protezione integrate nel codice . . . 62
5.2 Verifica dinamica . . . 62
5.2.1 Testing manuale dei flussi OID4VP . . . 62
5.2.2 Testing manuale dei flussi di errore . . . 63
5.3 Validazione tramite Conformance Test . . . 63
5.3.1 La Conformance Test Suite della OpenID Foundation . . . 63
5.3.2 Configurazione dell'ambiente di test . . . 63
5.3.3 Categorie di test eseguiti . . . 64
5.3.3.1 Test di flusso nominale . . . 64
5.3.3.2 Test negativi . . . 64
5.3.3.3 Verifica dei parametri di protocollo . . . 64

5.3.4 Risultati e iterazioni correttive 65
5.3.5 Valore della validazione tramite Conformance Test 65

6 Valutazione finale **67**
6.1 Sintesi e bilancio generale 67
6.2 Validazione analitica dei risultati 68
6.2.1 Obiettivi raggiunti 68
6.2.2 Debito tecnico e limiti 69
6.2.3 Sviluppi futuri e miglioramenti proposti 70
6.3 Bilancio delle competenze 70
6.3.1 Competenze tecniche (Hard Skills) 70
6.3.2 Competenze trasversali (Soft Skills) 71
6.4 Riflessione e chiusura 71

Bibliografia **73**

Elenco delle figure

1.1 Logo di Monokee 1

2.1 Diagramma dei casi d'uso – Scenario principale del Relying Party 8
2.2 UC1 – Inizializzazione della richiesta di presentazione 9
2.3 UC2 – Generazione del Request Object 13
2.4 UC3 – Ricezione ed analisi della risposta del Wallet 19
2.5 UC4 – Recupero della presentazione (fetch) 28
2.6 UC5 – Gestione del ciclo di vita del client 31
2.7 UC6 – Validazione della Trust Chain 34

3.1 Architettura generale del sistema Relying Party 44
3.2 Organizzazione della codebase 46

4.1 Flusso di inizializzazione della sessione di presentazione 54
4.2 Sessione salvata su Redis 55
4.3 Generazione del Request Object 56

Elenco delle tabelle

2.1 Requisiti funzionali 39
2.2 Requisiti qualitativi 41
2.3 Requisiti di vincolo 41
2.4 Riepilogo quantitativo dei requisiti 41

6.1 Stato di completamento dei requisiti funzionali. 68
6.2 Stato di completamento dei requisiti non funzionali. 69

Glossario

API in informatica con il termine *Application Programming Interface API* (ing. interfaccia di programmazione di un'applicazione) si indica ogni insieme di procedure disponibili al programmatore, di solito raggruppate a formare un set di strumenti specifici per l'espletamento di un determinato compito all'interno di un certo programma. La finalità è ottenere un'astrazione, di solito tra l'hardware e il programmatore o tra software a basso e quello ad alto livello semplificando così il lavoro di programmazione. 2

CI/CD acronimo di *Continuous Integration / Continuous Delivery*, indica l'insieme di pratiche di ingegneria del software che automatizzano l'integrazione frequente del codice, l'esecuzione dei test e il rilascio del software in ambienti di produzione. 3

CRUD acronimo di *Create, Read, Update, Delete*, indica le quattro operazioni fondamentali di gestione dei dati persistenti in un sistema informatico. 62

DCQL acronimo di *Digital Credentials Query Language*, è un linguaggio di interrogazione definito nell'ambito della specifica OpenID for Verifiable Presentations (OID4VP) che consente al Verifier di esprimere in modo strutturato e granulare i requisiti relativi alle credenziali verificabili richieste all'utente, specificando tipologie di credenziale, attributi da rivelare e autorità emittenti fidate. v, 9, 11, 12, 17, 20, 21, 24–27, 34, 39, 41, 45, 47, 48, 53, 56, 57, 65, 68

Digital Wallet applicazione software o servizio che consente a un utente di conservare, gestire e presentare in modo sicuro le proprie credenziali digitali (es. EUDI Wallet), agendo come tramite tra l'utente, le autorità emittenti (Issuer) e i fornitori di servizi (Verifier). 43, 44, 63, 65

DTO acronimo di *Data Transfer Object*, è un pattern architetturale che definisce un oggetto il cui unico scopo è il trasporto di dati tra i livelli di un'applicazione, privo di logica di business. 49

Forward Secrecy proprietà di un protocollo crittografico per cui la compromissione delle chiavi di lungo periodo non compromette la riservatezza delle sessioni di comunicazione precedentemente instaurate. Viene ottenuta mediante l'impiego di chiavi crittografiche effimere generate ad hoc per ciascuna sessione e distrutte al termine dell'utilizzo. v, 45, 50, 55, 57, 67

IAM acronimo di *Identity and Access Management*, indica l'insieme di processi, tecnologie e politiche volti a gestire le identità digitali e a regolamentare l'accesso alle risorse di un sistema informatico. 1

JAR acronimo di *JWT-Secured Authorization Request* (RFC 9101), è una specifica che richiede che la richiesta di autorizzazione OAuth 2.0 sia incapsulata in un JWT firmato digitalmente, garantendo l'integrità e l'autenticità dei parametri della richiesta. 53, 64

JWE acronimo di *JSON Web Encryption* (RFC 7516), è uno standard per la cifratura di contenuti in formato compatto e sicuro basato su JSON, utilizzato per proteggere la riservatezza dei dati trasmessi tra le parti di un protocollo crittografico. 3, 61, 63, 65

JWK acronimo di *JSON Web Key* (RFC 7517), è un formato standard per la rappresentazione di chiavi crittografiche in formato JSON. Un *JWK Set* (JWKS) è un insieme di chiavi JWK, tipicamente esposto tramite un endpoint pubblico per la distribuzione delle chiavi. 3

JWS acronimo di *JSON Web Signature* (RFC 7515), è uno standard per la firma digitale di contenuti in formato JSON, utilizzato per garantire l'integrità e l'autenticità dei dati trasmessi. 3, 61

JWT acronimo di *JSON Web Token* (RFC 7519), è un formato compatto e autonomo per la rappresentazione sicura di attestazioni (*claims*) tra due parti, tipicamente utilizzato per l'autenticazione e lo scambio di informazioni. 3, 63, 64

KB-JWT acronimo di *Key Binding JWT*, è un token JWT firmato dal dispositivo del titolare della credenziale (*Holder*) con la chiave privata associata alla credenziale SD-JWT. La sua verifica attesta il possesso effettivo della credenziale da parte del presentatore (*Proof of Possession*). 20, 21, 24, 25, 51, 56, 57, 63, 64, 67

Multi-tenancy architettura software in cui una singola istanza di un'applicazione serve molteplici organizzazioni o clienti (*tenant*), garantendo l'isolamento logico dei dati e delle configurazioni tra ciascun tenant pur condividendo la medesima infrastruttura. 43, 69

OAuth 2.0 framework di autorizzazione standardizzato (RFC 6749) che consente a un'applicazione di terze parti di ottenere un accesso limitato a un servizio HTTP, sia per conto di un proprietario della risorsa, sia per conto proprio. OID4VP si basa sulle estensioni di questo framework. 44, 59, 62

OID4VP acronimo di *OpenID for Verifiable Presentations*, è una specifica del framework OpenID che definisce il protocollo di interazione tra un Verifier (Relying Party) e un Digital Wallet per la richiesta, la trasmissione e la verifica di presentazioni di credenziali verificabili. 2, 43, 61–65

Relying Party nell'ambito dell'identità digitale, è l'entità o il servizio che si affida alle credenziali presentate da un utente e verificate tramite un'infrastruttura fidata per concedere l'accesso a una risorsa o fornire un servizio. vii, 1, 7–9, 28–30, 44, 48, 50, 51, 55, 67

Replay Attack tipologia di attacco informatico in cui un avversario intercetta e ritrasmette un messaggio o una credenziale legittima precedentemente catturata, al fine di ottenere accesso non autorizzato o di eseguire operazioni fraudolente. Le contromisure tipiche includono l'uso di *nonce* crittografici e di marcature temporali. 50, 51, 63, 64

SCIM acronimo di *System for Cross-domain Identity Management*, è un protocollo standardizzato (RFC 7643 e RFC 7644) per l'automazione dello scambio di informazioni di identità tra domini, che definisce schemi e API REST per la gestione del ciclo di vita delle risorse di identità in ambienti multi-tenant. 4, 7, 31–33, 48, 50, 53, 62, 68

SD-JWT acronimo di *Selective Disclosure JSON Web Token*, è un formato di credenziale verificabile basato su JWT che consente al titolare di rivelare selettivamente solo un sottoinsieme degli attributi contenuti nella credenziale, senza compromettere l'integrità crittografica dell'attestazione originaria emessa dall'Issuer. v, 2, 20–22, 24, 26, 27, 39, 41, 43, 45, 47, 48, 51, 53, 56, 63, 67

Trust Chain nel contesto di OpenID Federation 1.0, è una catena gerarchica di attestazioni digitali firmate che collega un'entità foglia (ad esempio un Issuer o un Verifier) a

un'ancora di fiducia (*Trust Anchor*) riconosciuta, permettendo di dedurre e verificare la legittimità dell'entità senza conoscenza diretta preventiva. v, vii, 2, 9–12, 26, 34–38, 40, 45, 50, 51, 54, 57, 67

TTL acronimo di *Time-To-Live*, indica il tempo massimo di validità o permanenza di un dato in un sistema di memorizzazione. In Redis, il TTL definisce il tempo trascorso il quale una chiave viene automaticamente rimossa dal datastore. 3

UML in ingegneria del software *UML, Unified Modeling Language* (ing. linguaggio di modellazione unificato) è un linguaggio di modellazione e specifica basato sul paradigma object-oriented. L'*UML* svolge un'importantissima funzione di "lingua franca" nella comunità della progettazione e programmazione a oggetti. Gran parte della letteratura di settore usa tale linguaggio per descrivere soluzioni analitiche e progettuali in modo sintetico e comprensibile a un vasto pubblico. 8

Verifier sinonimo di Relying Party nell'ecosistema OID4VP. Identifica il ruolo dell'entità che richiede una presentazione verificabile (*Verifiable Presentation*), ne valida l'autenticità e l'integrità crittografica prima di accettare le credenziali. 1, 2, 4, 7, 9–39, 45, 47–51, 53, 55, 57, 58, 61, 63–65

VP Token nel protocollo OID4VP, è il token che incapsula una o più presentazioni verificabili (*Verifiable Presentations*) trasmesse dal Wallet al Verifier in risposta a una richiesta di presentazione. Può contenere credenziali in formato SD-JWT, mdoc o altri formati supportati. v, 2, 7, 20, 21, 24–28, 30, 39, 40, 48, 51, 56, 57, 63, 64, 68

Acronimi e abbreviazioni

API Application Program Interface. 2, 7, 31–33, 41

BSON Binary JSON. 2, 50

CRUD Create, Read, Update, Delete. 68

DFS Depth-First Search. 57, 68
DTO Data Transfer Object. v, 49

HTTP HyperText Transfer Protocol. 2, 9, 44, 62, 65

IAM Identity and Access Management. 1, 2

JAR JWT-Secured Authorization Request. 53, 68
JWE JSON Web Encryption. v, 3, 13–16, 20–23, 40, 41, 47, 48, 50, 55–57, 59, 68
JWK JSON Web Key. 3, 17, 55, 71
JWKS JSON Web Key Set. 2, 3, 7, 11, 13, 15, 18, 19, 31, 34, 35, 37, 38, 47, 48, 55, 57, 58
JWS JSON Web Signature. 3, 41, 47, 48, 55, 68
JWT JSON Web Token. 2–4, 9–11, 13, 14, 17–27, 36, 38–41, 47, 51, 53–56, 58, 59, 68

mDoc Mobile Document. 70

OID4VP OpenID for Verifiable Presentations. v, 2, 7, 12, 14, 17, 39–41, 43–45, 47–49, 53–55, 59, 67
OIDF OpenID Foundation. v, 61, 63–65, 67, 69

REST Representational State Transfer. 44, 68

TTL Time-To-Live. 3, 4, 9, 12, 14–17, 27, 43, 49, 50, 54, 55, 58, 67

UML Unified Modeling Language. 8

Capitolo 1

Il contesto aziendale

1.1 Scelta dello stage

L'opportunità di svolgere lo stage aziendale è scaturita dalla partecipazione all'evento *Stage-IT 2026*, un'iniziativa promossa per favorire il confronto tra gli studenti e diverse realtà aziendali operanti in svariati ambiti del *software development*. Tra le diverse proposte presentate, l'attenzione si è focalizzata sul progetto offerto dall'azienda *Monokee*, incentrato sui temi dell'identità digitale e delle credenziali decentralizzate. Tale scelta è stata dettata da fattori chiave distinti: da un lato, il vivo interesse per gli aspetti innovativi e le sfide tecnologiche del progetto; dall'altro, la stretta coerenza con gli obiettivi accademici e professionali futuri, orientati al proseguimento degli studi nella laurea magistrale in cybersicurezza. L'analisi della documentazione tecnica preliminare fornita dall'azienda, unitamente ai colloqui conoscitivi effettuati, ha permesso di delineare nel dettaglio gli obiettivi del progetto, consolidando la decisione di svolgere il percorso di tirocinio presso Monokee.

Figura 1.1: Logo di Monokee
Fonte: www.monokee.com

1.2 Dominio applicativo

Monokee implementa una piattaforma avanzata focalizzata sui sistemi di *Identity and Access Management* (IAM_G) e sull'orchestrazione delle identità digitali. La soluzione aziendale propone un approccio innovativo che integra la gestione dell'identità con un orchestratore visivo *low-code/no-code*, consentendo alle organizzazioni di modellare, automatizzare e personalizzare flussi complessi di autenticazione e autorizzazione per utenti e dispositivi (*Identity Journeys*) senza la necessità di sviluppare codice ad hoc.

All'interno di questa architettura tecnologica, l'attività di stage si è focalizzata sulla progettazione e sullo sviluppo *ex novo* del modulo `monokee-relying-party`. Tale componente consente alla piattaforma Monokee di operare come *Relying Party*$_G$ (o *Verifier*$_G$) nell'ambito del paradigma delle credenziali decentralizzate e dell'ecosistema EUDI Wallet. Il modulo ha il compito di richiedere, ricevere e verificare le *Verifiable Credentials* presentate dai *wallet*

digitali degli utenti, convertendole in sessioni di accesso sicure e integrate nell'infrastruttura IAM aziendale.

Nello specifico, il componente si articola nelle seguenti funzionalità principali:

1. **Gestione delle *Presentation Requests*:** generazione e invio di richieste di presentazione standardizzate volte ad attestare specifici attributi dell'utente (ad esempio la maggiore età o la cittadinanza);
2. **Implementazione del protocollo OpenID4VP:** gestione completa del protocollo *OpenID for Verifiable Presentations* (OID4VP$_{\text{G}}$) per la ricezione e l'elaborazione sicura dei *Verifiable Presentation Tokens* (*VP Tokens*$_{\text{G}}$) tramite *endpoint* dedicati;
3. **Validazione delle prove crittografiche:** verifica dell'integrità e dell'autenticità delle attestazioni ricevute — comprese quelle strutturate secondo lo standard *Selective Disclosure JWT* (SD-JWT$_{\text{G}}$) — e convalida della catena di fiducia (*Trust Chain*$_{\text{G}}$) legata agli emettitori (*Issuers*).

1.3 Tecnologie utilizzate

1.3.1 TypeScript

TypeScript è un linguaggio di programmazione open-source sviluppato da Microsoft che estende JavaScript aggiungendo la tipizzazione statica opzionale e funzionalità avanzate come le interfacce, gli enum e i generics. TypeScript è stato scelto come linguaggio principale per lo sviluppo del microservizio, garantendo maggiore robustezza del codice, migliore esperienza di sviluppo grazie all'autocompletamento e al controllo dei tipi a tempo di compilazione, e una manutenibilità superiore rispetto a JavaScript puro.

1.3.2 Node.js

Node.js è un runtime JavaScript lato server che consente l'esecuzione di codice al di fuori del browser. Il progetto utilizza Node.js 24 come ambiente di esecuzione, sfruttandone il modello di I/O non bloccante e ad eventi per gestire in modo efficiente le richieste HTTP$_{\text{G}}$ concorrenti. Node.js rappresenta il fondamento su cui il microservizio viene costruito e distribuito, offrendo un ecosistema maturo e performante per applicazioni server-side.

1.3.3 Express.js

Express.js è un framework web minimalista e flessibile per Node.js, ampiamente adottato come standard de facto per la creazione di applicazioni server e API$_{\text{G}}$ RESTful$_{\text{G}}$. Nel progetto viene utilizzato come base per il server HTTP$_{\text{G}}$, gestendo il routing delle richieste, i middleware di elaborazione (pre e post processing) e la gestione degli errori. Il framework è integrato tramite le librerie proprietarie Monokee (monokee-base-server e monokee-scim-utils) che ne estendono le funzionalità per adattarlo alle esigenze specifiche della piattaforma.

1.3.4 MongoDB

MongoDB è un database NoSQL orientato ai documenti, che memorizza i dati in formato Binary JSON (BSON$_{\text{G}}$), offrendo flessibilità nello schema e scalabilità orizzontale nativa. Nel progetto viene utilizzato come database principale per la persistenza delle configurazioni dei client (Verifier), attraverso il repository MongoDBClientRepository che estende la classe base MongoDBBaseRepository fornita dalla libreria interna @monokee-dev/monokee-base-repository. MongoDB è stato scelto per la sua capacità di gestire documenti complessi e strutturati in modo naturale, come le configurazioni crittografiche (JWKS$_{\text{G}}$) dei Verifier OpenID4VP.

1.3.5 Redis

Redis è un data store in-memory open-source, utilizzato come database chiave-valore ad alte prestazioni, cache e message broker. Nel progetto viene impiegato tramite la libreria ioredis per la gestione delle sessioni di verifica con supporto a TTL_G (Time-To-Live) configurabile. La scelta di Redis è motivata dalla necessità di avere uno storage veloce e volatile per dati temporanei come le sessioni di presentazione.

1.3.6 jose (JavaScript Object Signing and Encryption)

jose è una libreria JavaScript/TypeScript per la gestione dei JSON Web Token (JWT_G), JSON Web Signature (JWS_G), JSON Web Encryption (JWE_G) e JSON Web Key (JWK_G), conforme alle specifiche RFC. Nel progetto rappresenta una dipendenza fondamentale, utilizzata estensivamente per: la firma e la verifica di JWT (richieste di autorizzazione e Entity Statement), la cifratura e decifratura JWE delle risposte del wallet, l'importazione e gestione delle chiavi crittografiche (JWK/JWKS) per molteplici algoritmi (ES256, RS256, ECDH-ES, ecc.), e la validazione dei certificati X.509 nella trust chain OpenID Federation. La libreria è alla base di tutta la logica crittografica del microservizio.

1.3.7 Docker

Docker è una piattaforma di containerizzazione che consente di impacchettare un'applicazione e tutte le sue dipendenze in un contenitore leggero e portabile. Il progetto utilizza un Dockerfile multi-stage: il primo stage (builder) compila il codice TypeScript e installa le dipendenze, mentre il secondo stage copia solamente gli artefatti necessari all'esecuzione (dist, node_modules, resources), riducendo significativamente la dimensione dell'immagine finale.

1.3.8 ESLint

ESLint è uno strumento di analisi statica del codice (linter) per JavaScript e TypeScript che identifica pattern problematici, errori e violazioni delle convenzioni di stile. Nel progetto è configurato con il parser @typescript-eslint/parser e integra diversi plugin: @typescript-eslint per regole specifiche TypeScript (come le naming convention per classi PascalCase e metodi camelCase), eslint-plugin-security per l'identificazione di potenziali vulnerabilità di sicurezza, e eslint-plugin-prettier per l'integrazione con il formattatore di codice. ESLint è eseguito sia in fase di sviluppo locale (tramite Nodemon) sia nella pipeline CI/CD_G.

1.3.9 Prettier

Prettier è un formattatore di codice che impone uno stile di formattazione consistente in modo automatico, eliminando i dibattiti sullo stile del codice all'interno del team. Nel progetto è configurato con opzioni specifiche: senza punto e virgola (semi: false), con singoli apici (singleQuote: true), virgole finali ovunque (trailingComma: all) e parentesi obbligatorie nelle arrow function (arrowParens: always). Prettier è integrato con ESLint tramite eslint-config-prettier per evitare conflitti tra le regole dei due strumenti.

1.3.10 Yarn

Yarn è un package manager per JavaScript sviluppato originariamente da Facebook, alternativo a npm, che offre installazione più veloce delle dipendenze, un lockfile deterministico (yarn.lock) e una gestione più affidabile della risoluzione dei pacchetti. Nel progetto Yarn viene utilizzato per la gestione di tutte le dipendenze, sia nella fase di sviluppo locale sia nella build Docker, ed è configurato per accedere al registry npm privato di GitHub Packages dell'organizzazione Monokee tramite il file .npmrc.

1.3.11 Nodemon

Nodemon è uno strumento di sviluppo per Node.js che monitora le modifiche ai file sorgente e riavvia automaticamente l'applicazione. Nel progetto è configurato per osservare le directory src, il file .env e package.json, monitorando le estensioni .js, .ts, .json, .yml, .yaml e .env. Ad ogni modifica, Nodemon esegue automaticamente la sequenza completa: installazione dipendenze, build TypeScript, audit di sicurezza, lint del codice ed esecuzione dell'applicazione tramite ts-node, garantendo un ciclo di sviluppo rapido e completo.

1.3.12 dotenv

dotenv è una libreria che carica le variabili d'ambiente da un file .env nell'oggetto process.env di Node.js, consentendo la separazione della configurazione dal codice sorgente. Nel progetto viene utilizzato per gestire la configurazione dell'applicazione, come gli URL di connessione a MongoDB e Redis, il livello di logging, i parametri TTL delle sessioni, gli URL dell'endpoint di autorizzazione OpenID4VCI e le credenziali del Verifier (Entity Identifier e Verifier Attestation JWT). Questo approccio segue il principio del Twelve-Factor App per la gestione delle configurazioni.

1.3.13 Librerie Interne Monokee

Il progetto si appoggia su un ecosistema di librerie proprietarie sviluppate internamente dall'organizzazione Monokee, distribuite come pacchetti npm privati tramite GitHub Packages. Le principali librerie utilizzate sono:

- @monokee-dev/monokee-base-server: fornisce la classe Server che incapsula la configurazione del server Express.js, la gestione delle rotte, i middleware e l'Async Local Storage per il contesto delle richieste.
- @monokee-dev/monokee-base-repository: mette a disposizione MongoDBBaseRepository, una classe base generica per l'interazione con MongoDB che implementa il pattern Repository.
- @monokee-dev/monokee-base-service: fornisce classi base per la logica di business dei servizi.
- @monokee-dev/monokee-base-filter: implementa i meccanismi di filtraggio per le query sui repository.
- @monokee-dev/monokee-scim-utils: offre utilità per il protocollo SCIM_{G} (System for Cross-domain Identity Management), inclusa la classe ScimServer e ScimBaseRoute utilizzate come base per il server e le rotte del progetto.
- @monokee-dev/monokee-logger: fornisce MonokeeLogger, un wrapper di logging strutturato con supporto a livelli configurabili e contesto di autenticazione.
- @monokee-dev/monokee-utils: raccoglie utilità trasversali, tra cui la gestione degli errori tipizzati (ForbiddenError, InvalidValueError, NotFoundError, PreconditionFailedError) e l'Async Local Storage (MonokeeAlsStore).
- @monokee-dev/monokee-health-probe: implementa gli endpoint di health check per il monitoraggio dello stato del microservizio negli ambienti containerizzati (readiness/liveness probe).

1.3.14 Azure Container Registry (ACR)

Azure Container Registry è il servizio di registry per immagini Docker offerto da Microsoft Azure. Nel progetto viene utilizzato come destinazione per la pubblicazione delle immagini Docker prodotte dalla pipeline CI/CD. Le immagini vengono taggate con la data di build e un prefisso dell'hash del commit Git, e pubblicate sul registry monokee4prod.azurecr.io,

pronte per essere distribuite negli ambienti di staging e produzione dell'infrastruttura cloud Monokee.

Capitolo 2

Analisi dei Requisiti

2.1 Panoramica e Attori

L'obiettivo dello stage è lo sviluppo di un modulo di relying party (come definito nella sezione 1.2) che si occuperà di creare e inviare richieste standardizzate per ottenere attributi specifici degli utenti, gestire la ricezione dei VP Tokens_{G} tramite endpoint dedicati e verificare l'integrità delle presentazioni e la validità degli emettitori.

2.1.1 Attori del sistema

Gli attori rappresentano le entità (utenti fisici o sistemi esterni) che interagiscono con l'applicazione. Per il progetto sviluppato, sono stati individuati i seguenti attori principali:

- **Relying Party:** rappresenta la componente backend del sistema che funge da entità richiedente e verificatrice delle Credenziali Verificabili. Gestisce l'intero ciclo di vita di una richiesta di presentazione: dall'inizializzazione della sessione, alla generazione del Request Object firmato crittograficamente, fino alla validazione delle presentazioni ricevute dal Wallet.
- **Wallet:** rappresenta un sistema esterno,tipicamente un'app mobile o un'estensione del browser, che custodisce le Credenziali Verificabili (Verifiable Credentials) emesse da uno o più Issuer fidati e le presenta su richiesta del Verifier. Nel contesto del protocollo OID4VP, il Wallet agisce come un intermediario tecnico tra l'Utente e il Relying Party: riceve la richiesta di presentazione, la interpreta, presenta all'Utente le credenziali richieste per ottenerne il consenso, e infine assembla e trasmette la Verifiable Presentation al Backend.
- **Utente finale:** è la persona fisica che interagisce con il sistema attraverso il Frontend del Relying Party e il proprio Wallet digitale. È il titolare delle Credenziali Verificabili, ovvero possiede documenti o attestazioni digitali (ad esempio un documento d'identità, una patente, un attestato professionale) emessi da Issuer fidati e memorizzati nel proprio Wallet. L'Utente rappresenta il soggetto che deve dimostrare il possesso di determinati attributi o qualifiche richieste dal servizio offerto dal Relying Party, ed è l'unico attore che può autorizzare la condivisione delle proprie credenziali.
- **Amministratore:** è l'operatore o il sistema autorizzato responsabile della gestione e della configurazione dei client (Verifier) all'interno della piattaforma attraverso le API SCIM. Governa l'intero ciclo di vita delle istanze dei client registrati nei singoli domini, definendone e aggiornandone i parametri operativi, i vincoli crittografici, gli URI di reindirizzamento autorizzati, i formati di *Verifiable Presentation* supportati e i relativi set di chiavi crittografiche (JWKS).

2.2 Casi d'uso

I diagrammi dei casi d'uso (*Use Case Diagram*) costituiscono una tipologia di diagrammi UML_G (*Unified Modelling Language*) finalizzata a descrivere le interazioni tra gli attori del sistema e le funzionalità che quest'ultimo è chiamato a fornire.

Nelle sottosezioni seguenti vengono descritti nel dettaglio i casi d'uso relativi all'integrazione del modulo Relying Party nel sistema. Ciascun caso d'uso è presentato secondo la seguente struttura standardizzata:

- **Caso d'uso:** codice identificativo e denominazione del caso d'uso;
- **Attori:** attori coinvolti nello scenario;
- **Descrizione:** descrizione dettagliata del caso d'uso, comprensiva dei passi che ne caratterizzano lo svolgimento;
- **Precondizioni:** condizioni che devono risultare soddisfatte affinché l'attore possa avviare il caso d'uso;
- **Postcondizioni:** stato del sistema al termine dell'esecuzione del caso d'uso;
- **Scenario principale:** sequenza ordinata di passi che descrivono il flusso nominale del caso d'uso, ossia la sequenza di azioni che si verifica in assenza di condizioni di errore;
- **Scenario alternativo:** eventuali flussi alternativi che possono verificarsi durante l'esecuzione del caso d'uso, con le relative condizioni di attivazione e le conseguenze sul sistema.

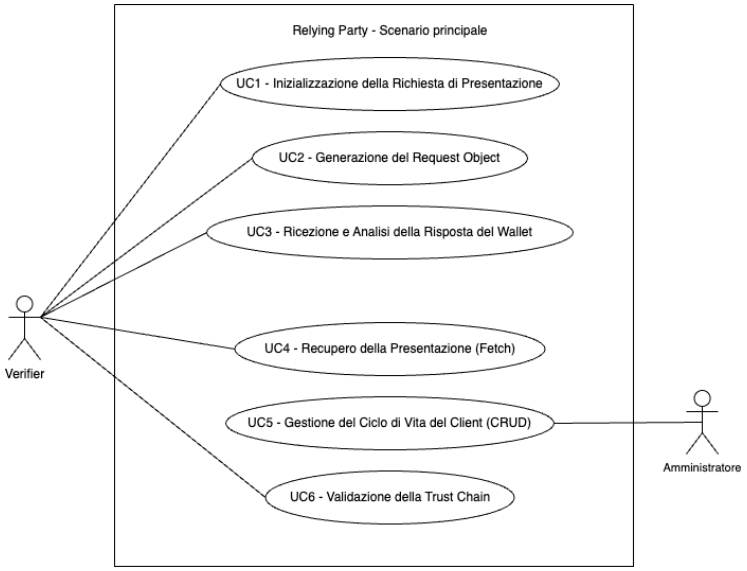


Figura 2.1: Diagramma dei casi d'uso – Scenario principale del Relying Party

UC1: Inizializzazione della richiesta di presentazione

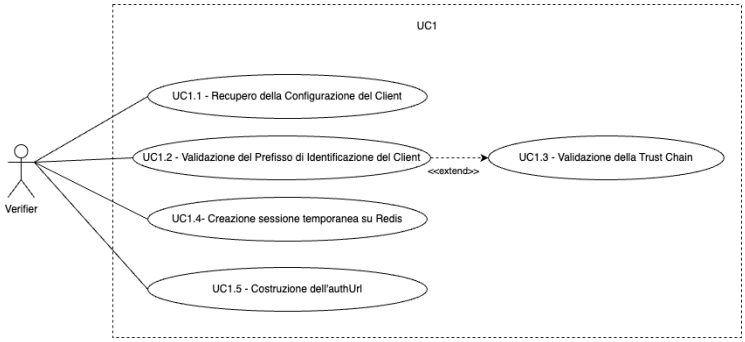


Figura 2.2: UC1 – Inizializzazione della richiesta di presentazione

Attori: Verifier, Utente finale.
Descrizione: L'Utente finale, mediante l'interfaccia *frontend* del Relying Party, avvia una nuova sessione di richiesta di presentazione di Credenziali Verificabili. Il Verifier procede alla validazione dell'identità del client precedentemente configurato, effettua — ove previsto dal *Client ID Scheme* adottato — la verifica della *Trust Chain*$_{\mathrm{G}}$ (nel caso di OpenID Federation), istanzia una sessione temporanea su Redis contenente la *DCQL Query*$_{\mathrm{G}}$, un *nonce* crittografico e un identificativo univoco, e restituisce all'interfaccia *frontend* un URL di autorizzazione (`authUrl`) e un identificativo di sessione (`sessionId`).
Precondizioni:

- Il client (Verifier) è stato precedentemente configurato nell'istanza MongoDB con parametri `redirectUris` e `vpFormatsSupported` validi.
- Il server Redis è raggiungibile e operativo.
- Le variabili d'ambiente del sistema sono configurate correttamente.
- Qualora il prefisso del `clientId` sia `openid_federation`, la variabile d'ambiente `VERIFIER_ENTITY_IDENTIFIER` deve risultare impostata e le *Trusted Authorities* devono essere raggiungibili.
- Qualora il prefisso sia `verifier_attestation`, la variabile d'ambiente `VERIFIER_ATTESTATION_JWT` deve contenere un token JWT di attestazione valido.

Postcondizioni:

- Una nuova sessione con stato `pending` è stata istanziata su Redis con un TTL (*Time To Live*) configurato.
- La sessione contiene i seguenti parametri: `sessionId`, `requestId`, `nonce`, `transactionId`, `dcqlQuery`, `responseMode`, `prefixClientId`, `responseUri`.
- L'interfaccia *frontend* ha ricevuto l'`authUrl` (contenente le informazioni necessarie al Wallet per elaborare la richiesta) e il `sessionId` (necessario per il successivo recupero della presentazione).

Scenario principale:

1. L'Utente, mediante l'interfaccia *frontend*, invia una richiesta HTTP$_{\mathrm{G}}$ `POST` all'endpoint `/init` con un payload contenente i parametri e `dcqlQuery`.
2. Il Verifier recupera la configurazione del client dall'istanza MongoDB tramite i parametri `domainId` e `clientId`.
3. Il Verifier valida il prefisso del `clientId`.
4. Qualora il prefisso sia `openid_federation`, il Client costruisce e valida la *Trust Chain* verso il *Trust Anchor* configurato.
5. Il Verifier genera un `nonce`, un `requestId`, un `transactionId` e un `sessionId`.
6. La sessione viene persistita su Redis.

7. Il Verifier costruisce l'`authUrl`.
8. Il Verifier restituisce una risposta HTTP `201 Created` con payload `{success: true, sessionId, authUrl}`.

Scenario alternativo:

- Qualora non risulti configurato alcun Verifier per il `domainId` fornito, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `403 Forbidden`.
- Qualora il `TrustAnchorService` non riesca a costruire la *Trust Chain*, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `403 Forbidden`.
- Qualora non sia configurato l'url per il flusso `openid_federation`, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora il prefisso fornito non sia valido, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `501 Not Implemented`.
- Qualora il prefisso sia `verifier_attestation` ma il token JWT non risulti valido, il sistema solleva un'eccezione. Viene restituita una risposta HTTP `400 Bad Request`.

UC1.1: Verifica dell'identità del client configurato

Attori: Verifier.
Descrizione: Il Verifier recupera dall'istanza MongoDB la configurazione del client associato ai parametri `domainId` e `clientId` ricevuti nella richiesta HTTP. Tale configurazione comprende i parametri necessari per l'intero flusso di presentazione: `redirectUris`, `vpFormatsSupported`, `jwks` e gli algoritmi crittografici supportati.
Precondizioni:

- I parametri `domainId` e `clientId` sono stati forniti nella richiesta HTTP.
- L'istanza MongoDB è raggiungibile e operativa.

Postcondizioni:

- L'oggetto `Client` è stato recuperato dall'istanza MongoDB.

Scenario principale:

1. Il Verifier recupera la configurazione del client dal repository MongoDB.
2. L'oggetto estratto dal repository viene validato e vengono verificati i campi obbligatori.

Scenario alternativo:

- Qualora nessun record corrisponda alla coppia `domainId`/`clientId` fornita, il sistema solleva un'eccezione e viene restituita una risposta HTTP `403 Forbidden`.

UC1.2: Validazione del prefisso di identificazione del client

Attori: Verifier.
Descrizione: Il Verifier effettua il *parsing* e la validazione del prefisso di identificazione del client (`prefixClientId`) al fine di determinare la modalità di autenticazione del Verifier nei confronti del Wallet.
Precondizioni:

- Il parametro `prefixClientId` è stato fornito nel corpo della richiesta HTTP; qualora risulti vuoto, viene applicato il valore predefinito `pre_registered`.
- La configurazione del client è stata recuperata con successo (rif. 2.2).

Postcondizioni:

- Il prefisso è stato ricondotto a uno dei seguenti valori: `redirect_uri`, `x509_san_dns`, `x509_hash`, `openid_federation`, `verifier_attestation`, `pre_registered`.
- Il risultato della validazione include i seguenti attributi: `clientIdValue`, `requiresSignature`, `headerExtensions`, `clientMetadataPolicy`.

Scenario principale:

1. Il Verifier effettua il *parsing* del valore del prefisso.
2. In base al prefisso riconosciuto, il sistema applica la corrispondente logica di validazione:
   - `redirect_uri`: il valore `clientIdValue` viene costruito concatenando il prefisso al `responseUri` nella forma `redirect_uri:<responseUri>`. Non è richiesta la firma crittografica del *Request Object*. La politica sui metadati del client è impostata a `MUST_INCLUDE`.
   - `x509_san_dns`: dal JWKS del client viene estratta la catena certificativa `x5c` e viene validato il certificato *leaf*. È richiesta la firma crittografica. La politica sui metadati è impostata a `MUST_INCLUDE`.
   - `x509_hash`: dal JWKS viene estratta la catena `x5c` e viene calcolato il digest SHA-256 del certificato *leaf*, utilizzato come identificativo del client. È richiesta la firma crittografica. La politica sui metadati è impostata a `MUST_INCLUDE`.
   - `openid_federation`: come identificativo viene adottato il valore dell'`entityIdentifier`. È richiesta la firma crittografica. I metadati del client non devono essere inclusi nella richiesta (`MUST_EXCLUDE`).
   - `verifier_attestation`: viene effettuato il *parsing* del JWT di attestazione del Verifier, dal quale vengono estratti i claim `sub` e `cnf`. Il sistema verifica inoltre che il `responseUri` sia presente nella lista `redirect_uris` contenuta nel JWT. È richiesta la firma crittografica. La politica sui metadati è impostata a `MUST_INCLUDE`.
   - `pre_registered`: viene utilizzato direttamente il `clientId` già registrato nel sistema. Non è richiesta la firma crittografica. L'inclusione dei metadati è facoltativa (`NOT_REQUIRED`).

Scenario alternativo:

- Qualora il prefisso fornito sia `origin` viene restituita una risposta HTTP `501 Not Implemented`.
- Per i prefissi `x509_san_dns` e `x509_hash`, qualora il JWKS del client non contenga una chiave di firma provvista di catena certificativa `x5c`, viene restituita una risposta HTTP `400 Bad Request`.
- Per il prefisso `x509_san_dns`, qualora nessun `dnsName` sia presente nel certificato corrisponda all'*hostname* del `responseUri`, viene restituita una risposta HTTP `403 Forbidden`.
- Per il prefisso `verifier_attestation`, qualora il JWT risulti privo dei claim `sub` o `cnf`, viene restituita una risposta HTTP `400 Bad Request`.
- Per il prefisso `verifier_attestation`, qualora il `responseUri` non sia incluso nella lista `redirect_uris` del JWT di attestazione, viene restituita una risposta HTTP `403 Forbidden`.

UC1.3: Validazione della Trust Chain

Attori: Verifier.
Descrizione: Qualora il prefisso di identificazione del client sia `openid_federation`, il sistema procede alla verifica della *Trust Chain* dell'OpenID Provider, preliminarmente alla generazione della sessione.
Precondizioni:

- Il prefisso identificato nel caso d'uso UC1.2 (rif. 2.2) è `openid_federation`.
- La variabile d'ambiente `OPENID4VCI_AUTHORIZE_URL` è configurata e l'endpoint corrispondente è raggiungibile.
- Almeno un *Trust Anchor* è specificato nel campo `trustedAuthorities` della *DCQL Query*.

Postcondizioni:

- L'appartenenza dell'OpenID Provider alla federazione di fiducia è stata confermata mediante la validazione completa della catena.

Scenario principale:

1. Il sistema verifica la presenza della variabile d'ambiente `OPENID4VCI_AUTHORIZE_URL` e ne estrae l'`entityId` dell'OpenID Provider.
2. Vengono estratte le `trustedAuthorities` dalla *DCQL Query* fornita dall'interfaccia *frontend*.
3. Il Verifier avvia la costruzione e la validazione della *Trust Chain* per l'`entityId` dell'OpenID Provider.

Scenario alternativo:

- Qualora la variabile d'ambiente non risulti configurata, viene restituita una risposta HTTP `400 Bad Request`.
- Qualora il `TrustAnchorService` rilevi una catena non valida, viene restituita una risposta HTTP `403 Forbidden`.

UC1.4: Creazione della sessione temporanea su Redis

Attori: Verifier.
Descrizione: Il Verifier genera l'insieme degli identificativi crittografici necessari per la sessione e ne effettua la persistenza su Redis con un TTL (*Time To Live*) configurabile.
Precondizioni:

- La validazione del prefisso è stata completata con successo (rif. 2.2).
- Qualora il prefisso sia `openid_federation`, la *Trust Chain* è stata validata (rif. 2.2).
- Il server Redis è raggiungibile e operativo.

Postcondizioni:

- Una nuova sessione con stato `pending` è stata persistita su Redis, indicizzata mediante il `requestId` come chiave primaria.
- Il TTL è stato impostato.

Scenario principale:

1. Il Verifier genera i seguenti identificativi crittografici:
   - `requestId`: utilizzato come parametro `state` nel protocollo OID4VP.
   - `nonce`: impiegato per la prevenzione degli attacchi di tipo *replay*.
   - `transactionId`: deputato al collegamento tra sessione e *Response Record*.
   - `sessionId`: restituito all'interfaccia *frontend* per il successivo recupero della presentazione.
2. Viene istanziato l'oggetto `Session` con validazione dei campi obbligatori e verifica della conformità dei valori `nonce` e `requestId`.
3. Il Verifier effettua la serializzazione dell'oggetto in formato JSON e ne esegue la persistenza su Redis con il TTL configurato.

Scenario alternativo:

- Qualora i valori `nonce` o `requestId` generati non superino la validazione di conformità viene restituita una risposta HTTP `400 Bad Request`.

UC1.5: Costruzione dell'Authorization URL

Attori: Verifier.
Descrizione: Il Verifier costruisce l'URL di autorizzazione (`authUrl`) destinato a essere restituito all'interfaccia *frontend* e successivamente presentato al Wallet mediante codice

QR o *deep link*.
Precondizioni:

- La sessione è stata creata con successo (rif. 2.2).
- Il risultato della validazione del prefisso è disponibile (rif. 2.2).

Postcondizioni:

- L'`authUrl` è stato costruito nel formato conforme al tipo di prefisso adottato.

Scenario principale:

1. Qualora la firma crittografica non sia richiesta: l'URL viene costruito utilizzando i seguenti *query parameters*: `client_id`, `response_type=vp_token`, `response_mode`, `response_uri`, `nonce`, `state` (corrispondente al `requestId`) e `dcql_query`. Qualora la `clientMetadataPolicy` sia impostata a `MUST_INCLUDE`, viene aggiunto il parametro `client_metadata` contenente `vpFormatsSupported` e, opzionalmente, `jwks`.
2. Qualora la firma crittografica sia richiesta: l'URL viene costruito includendo esclusivamente i parametri `client_id`, `request_uri` (puntante all'endpoint `/request/:requestId`) e `request_uri_method=post`.

UC2: Generazione del Request Object

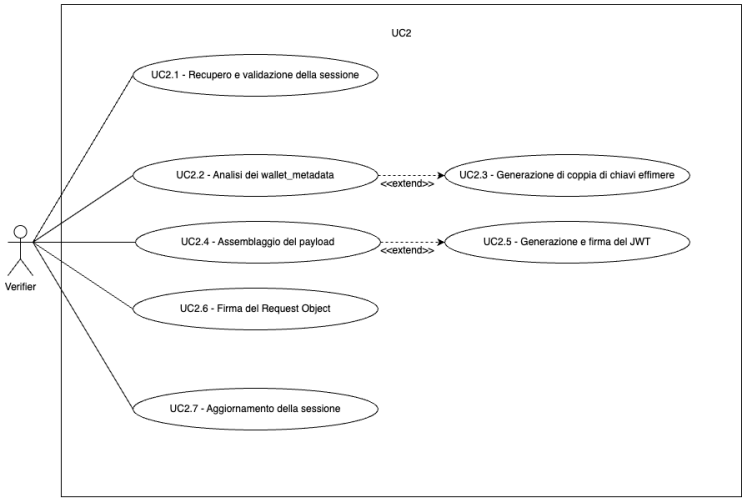


Figura 2.3: UC2 – Generazione del Request Object

Attori: Verifier, Wallet.
Descrizione: Il Wallet dell'Utente, dopo aver acquisito l'`authUrl` mediante scansione del codice QR o attivazione del *deep link*, contatta il Verifier al fine di ottenere il *Request Object* completo. Il Verifier recupera la sessione associata, negozia i parametri di sicurezza con il Wallet, quali gli algoritmi di cifratura JWE e gli algoritmi di firma, genera eventuali chiavi crittografiche effimere, assembla il *Request Object* sotto forma di JWT e lo restituisce al Wallet.
Precondizioni:

- La sessione corrispondente al `requestId` deve risultare presente su Redis e trovarsi in stato `pending`.
- Il *Request URI* non deve essere stato precedentemente consumato per la sessione in esame.
- Il client (Verifier) deve essere configurato nell'istanza MongoDB con un JWKS contenente la chiave privata di firma, qualora la firma sia richiesta dal prefisso adottato.

Postcondizioni:

- Il *Request Object* è stato generato sotto forma di JWT firmato crittograficamente.
- Qualora il Wallet supporti la cifratura JWE, una coppia di chiavi effimere è stata generata: la chiave pubblica è inclusa nel campo `client_metadata.jwks` del *Request Object*, mentre la chiave privata è stata persistita su Redis.
- La sessione è stata aggiornata in modo da impedire il riutilizzo del *Request URI*.

Scenario principale:

1. Il Wallet invia una richiesta HTTP `POST` (o `GET`) contenente, opzionalmente, i parametri `wallet_nonce` e `wallet_metadata` nel corpo della richiesta.
2. Il Verifier recupera la sessione da Redis mediante il `requestId`.
3. Il Verifier verifica che il *Request URI* non sia stato precedentemente consumato.
4. Il Verifier analizza i metadati del Wallet (`wallet_metadata`), procedendo alla negoziazione di alcuni parametri.
5. Qualora sia necessaria la cifratura JWE, il Verifier genera una coppia di chiavi effimere, persiste la chiave privata su Redis con TTL configurato e include la chiave pubblica nel campo `client_metadata.jwks`.
6. Il Verifier assembla il payload del *Request Object* conformemente alla specifica OID4VP.
7. Qualora il prefisso sia `openid_federation`, il Verifier genera e firma un *Entity Configuration* JWT con le proprie chiavi crittografiche, includendolo nel campo `trust_chain`.
8. Il Verifier appone la firma crittografica al *Request Object* mediante la propria chiave privata.
9. La sessione viene aggiornata impostando il flag `isRequestUriConsumed` al valore `true`.
10. Il Verifier restituisce il JWT con *content-type* `application/oauth-authz-req+jwt` e codice di stato HTTP `200 OK`.

Scenario alternativo:

- Qualora la sessione associata al `requestId` non risulti presente su Redis (ad esempio per scadenza del TTL), il sistema restituisce una risposta HTTP `404 Not Found`.
- Qualora il Wallet tenti di recuperare il *Request Object* per una sessione il cui *Request URI* sia stato già consumato, il sistema restituisce una risposta HTTP `403 Forbidden`. Tale meccanismo previene gli attacchi di tipo *replay*.
- Qualora il Wallet e il Verifier non dispongano di algoritmi JWE comuni, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora i formati *Verifiable Presentation* supportati dal Wallet non presentino alcuna intersezione con quelli del Verifier, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora l'algoritmo di firma indicato dal Wallet non sia compreso tra quelli supportati dal Verifier, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora il prefisso richieda la firma crittografica ma il JWKS del client non contenga chiavi private, il sistema restituisce una risposta HTTP `400 Bad Request`.
- Qualora la chiave privata non corrisponda al certificato *leaf* nella catena `x5c`, il sistema restituisce una risposta HTTP `400 Bad Request`.

UC2.1: Recupero e validazione della sessione

Attori: Verifier, Wallet.
Descrizione: Il Verifier recupera la sessione da Redis mediante il `requestId` fornito dal Wallet e ne verifica lo stato, accertando che il *Request URI* non sia stato precedentemente consumato. Tale verifica è finalizzata alla prevenzione di richieste duplicate e di attacchi di tipo *replay*.
Precondizioni:

- Il Wallet ha estratto il `requestId` dall'`authUrl` ricevuto.

Postcondizioni:

- La sessione è stata recuperata da Redis e risulta valida (non scaduta e non precedentemente consumata).

Scenario principale:

1. Il componente `SessionService` effettua la ricerca della sessione su Redis mediante il `requestId`.
2. Il sistema verifica che il flag `isRequestUriConsumed` risulti impostato al valore `false`.

Scenario alternativo:

- Qualora il TTL della sessione su Redis sia scaduto, il sistema solleva un'eccezione di tipo `NotFoundError`. Viene restituita una risposta HTTP `404 Not Found`.
- Qualora il flag `isRequestUriConsumed` risulti impostato al valore `true`, il sistema solleva un'eccezione di tipo `ForbiddenError`. Viene restituita una risposta HTTP `403 Forbidden`.

UC2.2: Analisi dei metadati del Wallet

Attori: Verifier.
Descrizione: Il Verifier analizza i metadati del Wallet (`wallet_metadata`) ricevuti nella richiesta HTTP `POST` e li confronta con la propria configurazione al fine di selezionare gli algoritmi crittografici comuni per la cifratura JWE della risposta, la firma del *Request Object* e i formati *Verifiable Presentation* supportati.
Precondizioni:

- La sessione risulta valida (rif. 2.2).
- La configurazione del client è stata recuperata con successo (rif. 2.2).

Postcondizioni:

- Sono stati selezionati, ove disponibili, i seguenti parametri: l'algoritmo JWE (`alg`), l'algoritmo di cifratura del contenuto (`enc`) e l'algoritmo di firma (`signingAlg`).

Scenario principale:

1. Il Verifier effettua il *parsing* dei `wallet_metadata` (da stringa JSON o oggetto strutturato).
2. Qualora i metadati risultino assenti o vuoti, vengono adottati i valori predefiniti dalla configurazione del Verifier.
3. Qualora i metadati siano presenti, il sistema procede alla ricerca dell'intersezione tra gli algoritmi supportati:
   - selezione dell'algoritmo `jweAlg`; qualora sia stato individuato un `jweAlg` ma non un corrispondente `enc`, viene tentato il *fallback* all'algoritmo `A128GCM`;
   - selezione dell'algoritmo `enc`;
   - selezione dell'algoritmo `signingAlg`.
4. Il sistema verifica la compatibilità dei formati *Verifiable Presentation*.

Scenario alternativo:

- Qualora non esista alcun algoritmo `alg` comune tra Wallet e Verifier, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora non esista alcun algoritmo `enc` comune, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora non esista alcun algoritmo di firma comune, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora non esista alcun formato *Verifiable Presentation* comune, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora sia stato negoziato esclusivamente il parametro `alg` o esclusivamente il parametro `enc` (ma non entrambi), il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.

UC2.3: Generazione della coppia di chiavi effimere

Attori: Verifier.
Descrizione: Il Verifier genera una coppia di chiavi crittografiche effimere destinate alla cifratura JWE della risposta del Wallet. La chiave pubblica viene inclusa nel *Request Object* affinché il Wallet possa cifrare la propria risposta; la chiave privata viene persistita su Redis per consentire la successiva decifratura da parte del Verifier.
Precondizioni:

- L'algoritmo JWE è stato negoziato con successo nel caso d'uso UC2.2 (rif. 2.2).
- La `clientMetadataPolicy` è impostata al valore `MUST_INCLUDE`.

Postcondizioni:

- Una coppia di chiavi effimere (pubblica/privata) è stata generata con un identificativo `kid` univoco in formato UUID.
- La chiave privata è stata persistita su Redis con un TTL configurato.
- La chiave pubblica è disponibile per l'inclusione nel *Request Object*.

Scenario principale:

1. Il Verifier determina la tipologia di chiave crittografica in funzione dell'algoritmo JWE negoziato.
2. La coppia di chiavi viene generata.

3. Le chiavi vengono esportate in formato JWK (*JSON Web Key*).
4. La chiave privata in formato JWK viene serializzata in JSON e persistita su Redis con il TTL configurato.

Scenario alternativo:

- Qualora l'algoritmo negoziato non appartenga alle famiglie ECDH o RSA, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.

UC2.4: Assemblaggio del payload del Request Object

Attori: Verifier.
Descrizione: Il Verifier assembla il payload del *Request Object* includendo tutti i parametri richiesti dalla specifica OID4VP. Il payload comprende i parametri della sessione, la *DCQL Query* e, opzionalmente, i `client_metadata`, contenenti le chiavi effimere e i formati *Verifiable Presentation* supportati e il campo `trust_chain` nel caso di OpenID Federation.
Precondizioni:

- La sessione risulta valida (rif. 2.2).
- L'analisi dei `wallet_metadata` è stata completata (rif. 2.2).
- Ove applicabile, le chiavi effimere sono state generate (rif. 2.2).

Postcondizioni:

- Il payload del *Request Object* è stato assemblato come oggetto strutturato contenente tutti i campi richiesti dal protocollo.

Scenario principale:

1. Il Verifier compone il payload includendo i seguenti claim obbligatori:
   - `iss`: identificativo del Verifier (`clientIdValue` derivato dal risultato della validazione del prefisso);
   - `aud`: destinatario (*audience*) del token;
   - `iat`: *timestamp* di emissione;
   - `exp`: *timestamp* di scadenza;
   - `client_id`: identificativo del client;
   - `response_type`: impostato al valore `vp_token`;
   - `response_mode`: modalità di risposta (`direct_post` o `direct_post.jwt`);
   - `response_uri`: URI sul quale il Wallet trasmetterà la risposta;
   - `nonce`: *nonce* crittografico della sessione;
   - `state`: corrispondente al `requestId` della sessione;
   - `dcql_query`: *DCQL Query* serializzata.
2. Il Verifier aggiunge i claim opzionali in funzione del contesto operativo:
   - `wallet_nonce`: incluso qualora il Wallet abbia fornito un *nonce* nella richiesta HTTP `POST`;
   - `client_metadata`: incluso qualora la `clientMetadataPolicy` sia impostata a `MUST_INCLUDE`.
3. Il Verifier costruisce l'header JWT con i seguenti campi: `typ` impostato a `oauth-authz-req+jwt`, `alg` corrispondente all'algoritmo di firma negoziato, e le eventuali *header extensions* specifiche del prefisso.

UC2.5: Generazione e firma del JWT dell'Entity Configuration

Attori: Verifier.
Descrizione: Qualora il prefisso di identificazione del client sia `openid_federation`, il Verifier genera un *Entity Configuration* JWT firmato con le proprie chiavi crittografiche. Tale JWT viene incluso nel campo `trust_chain` del payload del *Request Object*, consentendo

al Wallet di verificare l'identità del Verifier nell'ambito della federazione di fiducia.
Precondizioni:

- Il prefisso di identificazione del client è `openid_federation`.
- La variabile `VERIFIER_ENTITY_IDENTIFIER` è configurata.
- Il JWKS del client contiene almeno una chiave privata idonea alla firma crittografica.

Postcondizioni:

- Un *Entity Configuration* JWT è stato generato e firmato crittograficamente con l'algoritmo negoziato.
- Il JWT è stato inserito nel campo `trust_chain` del payload assemblato (rif. 2.2).

Scenario principale:

1. Il Verifier estrae la chiave privata dal JWKS del client e ne deriva la corrispondente chiave pubblica.
2. Il Verifier assembla il payload dell'*Entity Configuration* conformemente alla specifica OpenID Federation.
3. Il JWT viene firmato crittograficamente con la chiave privata del Verifier.
4. Il JWT firmato viene inserito nell'array `trust_chain` del payload del *Request Object*.

Scenario alternativo:

- Qualora il JWKS non contenga alcuna chiave, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.

UC2.6: Firma del Request Object

Attori: Verifier.
Descrizione: Il Verifier appone la firma crittografica al payload assemblato (rif. 2.2) sotto forma di JWT, utilizzando la chiave privata del client e l'algoritmo negoziato. Per i prefissi basati su certificati X.509, viene preliminarmente verificata la corrispondenza tra la chiave privata e il certificato *leaf* nella catena `x5c`. Qualora la firma non sia richiesta dal prefisso, il JWT viene serializzato in modalità *unsigned*.
Precondizioni:

- Il payload del *Request Object* è stato completamente assemblato (rif. 2.2), includendo eventualmente il campo `trust_chain` (rif. 2.2).
- L'header JWT è stato costruito con i campi `typ`, `alg` e le eventuali *header extensions* specifiche del prefisso.

Postcondizioni:

- Il *Request Object* JWT è stato generato, firmato crittograficamente o in modalità *unsigned*, ed è pronto per essere restituito al Wallet.

Scenario principale:

1. Qualora la firma crittografica sia richiesta (prefissi `x509_san_dns`, `x509_hash`, `openid_federation`, `verifier_attestation`):
   - il Verifier importa la chiave privata dal JWKS del client;
   - per i prefissi `x509_san_dns` e `x509_hash`, il sistema verifica la corrispondenza tra la chiave privata e la chiave pubblica del certificato *leaf* nella catena `x5c`.
2. Qualora la firma crittografica non sia richiesta (prefissi `redirect_uri`, `pre_registered`):
   - il JWT viene serializzato in modalità *unsigned* (`alg:none`), con header e payload codificati in `base64url` e campo firma vuoto.

Scenario alternativo:

- Qualora il JWKS non contenga alcuna chiave e la firma crittografica sia richiesta, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora i parametri della chiave privata non corrispondano a quelli del certificato *leaf*, il sistema solleva un'eccezione di tipo `InvalidValueError`. Viene restituita una risposta HTTP `400 Bad Request`.
- Qualora un prefisso non standard richieda la firma ma l'attributo `requiresSignature` risulti impostato a `false`, il sistema solleva un'eccezione di tipo `PreconditionFailedError`. Viene restituita una risposta HTTP `412 Precondition Failed`.

UC2.7: Aggiornamento della sessione

Attori: Verifier.
Descrizione: A seguito della generazione del *Request Object*, il Verifier aggiorna la sessione su Redis impostando il flag `isRequestUriConsumed` al valore `true`. Tale meccanismo garantisce che il *Request URI* possa essere utilizzato una sola volta, prevenendo attacchi di tipo *replay* e richieste duplicate da parte del Wallet.
Precondizioni:

- Il *Request Object* JWT è stato generato con successo (rif. 2.2).
- La sessione risulta presente su Redis.

Postcondizioni:

- Il flag `isRequestUriConsumed` della sessione è stato impostato al valore `true` su Redis.
- Qualsiasi richiesta successiva recante il medesimo `requestId` verrà rifiutata dal sistema.

Scenario principale:

1. Il Verifier aggiorna la sessione su Redis impostando il flag `isRequestUriConsumed` al valore `true`.
2. Il Verifier restituisce il *Request Object* JWT al Wallet nella risposta HTTP con codice di stato `200 OK`.

UC3: Ricezione ed analisi della risposta del Wallet

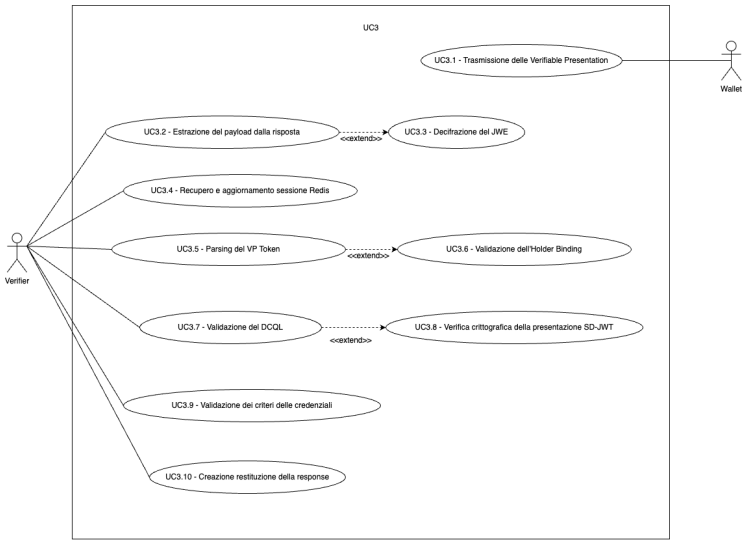


Figura 2.4: UC3 – Ricezione ed analisi della risposta del Wallet

Attori: Verifier, Wallet.
Descrizione: A seguito dell'ottenimento del consenso esplicito da parte dell'Utente, il Wallet inoltra la *Verifiable Presentation* al Verifier. Quest'ultimo esegue, ove necessario, la decifratura del payload JWE, la validazione della firma crittografica apposta sull'*Issuer* JWT e delle corrispondenti *disclosure* SD-JWT, la verifica del *nonce* e del *Key Binding* JWT ai fini della prevenzione di attacchi di tipo *replay*, nonché il controllo di conformità rispetto ai criteri stabiliti nella *DCQL Query*. Al termine del processo di validazione, il Verifier persiste il *VP Token* validato su Redis all'interno di un *Response Record* associato a un codice monouso (`responseCode`), restituendo al Wallet un URI di reindirizzamento che incorpora il parametro `response_code`.
Precondizioni:

- Il Wallet ha ricevuto e processato con esito positivo il *Request Object* (rif. 2.2).
- L'Utente ha espresso il proprio consenso alla presentazione delle credenziali richieste.
- La sessione è presente su Redis nello stato `pending`, risultando non scaduta e non precedentemente consumata.
- Il parametro `state` incluso nella risposta del Wallet corrisponde al `requestId` di una sessione attiva.

Postcondizioni:

- Lo stato della sessione su Redis è stato aggiornato a `response_received`.
- È stato istanziato e persistito su Redis un `ResponseRecord` in stato `pending_fetch`, contenente il *VP Token* validato e un identificativo univoco `responseCode`.
- Il Wallet ha ricevuto un `redirect_uri` recante il `response_code` quale frammento dell'URL.

Scenario principale:

1. Il Wallet trasmette una richiesta HTTP `POST` all'endpoint, includendo nel payload i parametri `vp_token`, `state` e, in via opzionale, `presentation_submission`.
2. Nel caso in cui sia presente il parametro `response`:
   - il Verifier determina se il payload costituisca un token JWE;
   - qualora venga identificato un token JWE, il Verifier recupera la chiave privata effimera da Redis tramite il `kid` specificato nell'header JWE, procede alla decifratura del messaggio e provvede alla rimozione immediata della chiave privata dal database;
   - il payload decifrato viene sottoposto ad analisi sintattica (*parsing*) nella forma di oggetto JSON o di token JWT;
   - dal payload risultante vengono estratti i parametri `state`, `vp_token`, `error` ed `error_description`.
3. Il Verifier accerta la presenza obbligatoria del parametro `state`.
4. Il Verifier recupera la sessione da Redis in base al valore di `state` e ne aggiorna lo stato a `response_received`.
5. Il Verifier esegue il *parsing* del parametro `vp_token`.
6. Il Verifier valida la conformità della presentazione rispetto ai vincoli definiti nella *DCQL Query* della sessione.
7. Nel caso in cui sia presente il *Key Binding* JWT (KB-JWT):
   - viene verificata la firma del KB-JWT mediante la chiave pubblica dell'*Issuer* JWT;
   - viene accertato che il parametro `nonce` contenuto nel KB-JWT coincida con il *nonce* della sessione, al fine di garantire la protezione contro attacchi di tipo *replay*;

- viene verificato che il claim `aud` (*audience*) corrisponda al `clientIdValue` del Verifier;
- viene controllato che il claim `sd_hash` presente nel KB-JWT corrisponda al digest calcolato sulla presentazione SD-JWT.

8. Nel caso in cui sia prescritto l'obbligo di *Holder Binding* crittografico ma il KB-JWT risulti assente, il sistema genera un'eccezione.
9. Il Verifier istanzia su Redis un `ResponseRecord` contenente il *VP Token* validato e il codice `responseCode` generato.
10. Il Verifier restituisce una risposta HTTP `200 OK` il cui payload JSON contiene il parametro `redirect_uri`.

Scenario alternativo:

- Nel caso in cui il Wallet notifichi una condizione di errore mediante il parametro `error` in luogo del `vp_token`, il Verifier recupera la sessione tramite lo `state`, ne imposta lo stato a `failed` e restituisce un `redirect_uri` contenente i parametri di errore.
- Nel caso in cui la sessione non risulti presente su Redis o non si trovi nello stato `pending`, il sistema solleva un'eccezione di tipo `NotFoundError`.
- Nel caso in cui il parametro obbligatorio `state` risulti assente dalla risposta, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui risultino assenti sia il parametro `vp_token` sia il parametro `error`, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui la verifica crittografica della firma dell'*Issuer* JWT non abbia esito positivo, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui il *nonce* contenuto nel *Key Binding* JWT non coincida con il *nonce* memorizzato nella sessione, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui il valore del claim `sd_hash` nel KB-JWT non corrisponda al digest calcolato sulla presentazione, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui le credenziali presentate non soddisfino i criteri definiti nella *DCQL Query*, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il digest di una *disclosure* non figuri tra gli hash contenuti nell'array `_sd` del payload dell'*Issuer*, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui la decifratura del payload JWE non abbia esito positivo o la chiave privata effimera non sia reperibile su Redis, il sistema solleva un'eccezione.
- Nel caso in cui la *Credential Query* prescriva il vincolo di *Holder Binding* ma la presentazione risulti priva del *Key Binding* JWT, il sistema solleva un'eccezione di tipo `PreconditionFailedError`.

UC3.1: Trasmissione della Verifiable Presentation

Attori: Wallet, Verifier.
Descrizione: Il Wallet inoltra al Verifier la risposta contenente la *Verifiable Presentation* mediante una richiesta HTTP `POST` all'endpoint designato. La trasmissione può avvenire secondo la modalità `direct_post`, con parametri nel corpo della richiesta, oppure secondo la modalità `direct_post.jwt`, in cui un parametro `response` contiene un token JWT o un token JWE cifrato che incapsula i parametri della risposta. Il payload comprende il parametro `state`, preposto alla correlazione della risposta con la sessione di origine, il *VP Token*, costituito dalle credenziali verificabili oggetto di presentazione, oppure, in caso di esito negativo, i relativi codici di errore.
Precondizioni:

- Il Wallet ha ricevuto e processato il *Request Object* (rif. 2.2).
- L'Utente ha autorizzato la condivisione delle credenziali richieste attraverso l'interfaccia del Wallet.

- L'endpoint del Verifier è raggiungibile e operativo.

Postcondizioni:

- La richiesta HTTP `POST` è stata ricevuta dal Verifier unitamente al corpo contenente i dati della presentazione.

Scenario principale:

1. Il Wallet compone il corpo della richiesta includendo i parametri `state`, corrispondente al `requestId` ricevuto nel *Request Object*, e `vp_token`, oggetto JSON contenente le presentazioni in formato SD-JWT.
2. Nel caso in cui la modalità negoziata sia `direct_post.jwt` e il Wallet supporti la cifratura, i parametri vengono incapsulati all'interno di un token JWE cifrato con la chiave pubblica effimera fornita nel *Request Object*.
3. Il Wallet trasmette la richiesta HTTP `POST` all'endpoint specificato nel parametro `response_uri` del Verifier.

Scenario alternativo:

- Nel caso in cui il Wallet non sia in grado di soddisfare la richiesta, esso trasmette un payload contenente i parametri `state` ed `error` in luogo del `vp_token`.

UC3.2: Estrazione del payload dalla risposta

Attori: Verifier.
Descrizione: Il Verifier esamina il corpo della richiesta HTTP `POST` pervenuta dal Wallet allo scopo di estrarre i parametri rilevanti (`state`, `vp_token`, `error`, `error_description`).
Precondizioni:

- Il Wallet ha inoltrato la risposta di presentazione (rif. 2.2).

Postcondizioni:

- I parametri `state`, `vp_token` (ovvero `error` ed `error_description`) sono stati estratti dal payload e resi disponibili per le fasi successive di validazione.

Scenario principale:

1. Il Verifier accerta la presenza del parametro `response` nel corpo della richiesta HTTP.
2. Nel caso in cui il parametro `response` sia assente (modalità `direct_post`), i parametri `state`, `vp_token`, `error` ed `error_description` vengono acquisiti direttamente dal corpo della richiesta.
3. Nel caso in cui il parametro `response` sia presente (modalità `direct_post.jwt`):
   - il Verifier determina se il valore costituisca un token JWE; in caso affermativo, invoca la procedura di decifratura (rif. 2.2);
   - la stringa in chiaro risultante viene sottoposta ad analisi sintattica, dapprima come oggetto JSON e, in caso di mancata corrispondenza, come token JWT;
   - i parametri `state`, `vp_token`, `error` ed `error_description` vengono estratti dal payload decodificato.

Scenario alternativo:

- Nel caso in cui il parametro obbligatorio `state` risulti assente al termine della fase di estrazione, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui risultino simultaneamente assenti sia il parametro `vp_token` sia il parametro `error`, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC3.3: Decifratura del JWE

Attori: Verifier.
Descrizione: Il Verifier esegue la decifratura del token JWE in formato compatto contenuto nel campo `response`, impiegando la chiave privata effimera generata nel caso d'uso UC2.3

(rif. 2.2) e precedentemente persistita su Redis. Al fine di preservare la proprietà di *forward secrecy*, la chiave viene rimossa da Redis in modo atomico contestualmente al suo recupero.
Precondizioni:

- Il campo `response` contiene una stringa JWE sintatticamente valida.
- La chiave privata effimera è presente nell'istanza Redis.

Postcondizioni:

- Il testo in chiaro del JWE è stato estratto e decodificato come stringa.
- La chiave privata effimera è stata rimossa da Redis mediante un'operazione atomica.

Scenario principale:

1. Il Verifier decodifica l'header protetto del JWE ed estrae il parametro `kid`.
2. Il Verifier recupera la chiave privata effimera da Redis, acquisendone il valore e rimuovendola contestualmente dal datastore.
3. La chiave privata viene importata nel formato crittografico appropriato.
4. Il token JWE viene sottoposto a decifratura.
5. Il payload in chiaro viene decodificato come stringa.

Scenario alternativo:

- Nel caso in cui l'header protetto del JWE sia privo del parametro `kid`, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui la chiave privata effimera non sia presente in Redis, il sistema solleva un'eccezione di tipo `NotFoundError`.
- Nel caso in cui l'operazione di decifratura crittografica non abbia esito positivo, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC3.4: Recupero e aggiornamento della sessione su Redis

Attori: Verifier.
Descrizione: Il Verifier recupera la sessione da Redis mediante il valore di `state`, corrispondente al `requestId`, estratto dal payload della risposta, e ne aggiorna lo stato in modo atomico da `pending` a `response_received`. Tale transizione atomica garantisce che per ciascuna sessione attiva possa essere accettata ed elaborata una sola risposta.
Precondizioni:

- Il parametro `state` è stato estratto con successo dal payload (rif. 2.2).
- Non è stata rilevata alcuna condizione di errore segnalata dal Wallet (il parametro `error` risulta assente).

Postcondizioni:

- La sessione è stata recuperata da Redis con l'insieme dei parametri necessari alle successive fasi di validazione.
- Lo stato della sessione su Redis è stato aggiornato atomicamente a `response_received`.

Scenario principale:

1. Il Verifier accerta atomicamente che la sessione si trovi nello stato `pending` e ne aggiorna lo stato a `response_received`.
2. Il Verifier recupera la configurazione del client dall'istanza MongoDB sulla base dei parametri `domainId` e `clientId` associati alla sessione.
3. Il Verifier ricalcola i parametri di identificazione del client a partire dal `prefixClientId` della sessione, necessari per le successive verifiche di *audience* del *Key Binding* JWT.

Scenario alternativo:

- Nel caso in cui la sessione non risulti presente su Redis o non si trovi nello stato `pending`, il sistema solleva un'eccezione di tipo `NotFoundError`.

- Nel caso in cui la configurazione del client non sia più reperibile nell'istanza MongoDB, il sistema solleva un'eccezione di tipo `NotFoundError`.

UC3.5: Parsing e validazione strutturale del VP Token

Attori: Verifier.
Descrizione: Il Verifier esegue il *parsing* del parametro `vp_token` ricevuto dal Wallet e ne verifica la conformità strutturale. Il *VP Token* deve essere rappresentato come un oggetto JSON le cui chiavi corrispondono agli identificativi delle query di credenziale (`credentialQueryId`) specificati nella *DCQL Query*, e i cui valori sono costituiti da array di stringhe rappresentanti le credenziali in formato SD-JWT.
Precondizioni:

- Il parametro `vp_token` è stato estratto dal payload (rif. 2.2).
- La sessione è stata recuperata con successo (rif. 2.2).

Postcondizioni:

- Il *VP Token* è stato deserializzato in un oggetto strutturato di tipo `VpToken`.

Scenario principale:

1. Il Verifier avvia l'analisi sintattica del parametro `vp_token`.
2. Nel caso in cui il parametro `vp_token` sia fornito come stringa, ne viene eseguito il *parsing* JSON.
3. Il sistema verifica che la struttura risultante costituisca un oggetto valido.
4. Per ciascuna voce dell'oggetto, il sistema accerta che il valore associato sia un array non vuoto di stringhe.

Scenario alternativo:

- Nel caso in cui il *parsing* JSON non abbia esito positivo a causa di una stringa malformata, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il *VP Token* non rispetti i vincoli strutturali attesi, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC3.6: Validazione dell'Holder Binding

Attori: Verifier.
Descrizione: Il Verifier verifica che il soggetto presentatore della credenziale (*Holder*) dimostri il legittimo possesso della chiave privata associata alla credenziale medesima, prevenendo in tal modo l'utilizzo fraudolento di credenziali sottratte o illecitamente duplicate.
Precondizioni:

- La presentazione SD-JWT include un *Key Binding* JWT.
- La verifica crittografica della presentazione SD-JWT è stata completata con esito positivo (rif. 2.2), rendendo disponibile il claim `cnf.jwk` all'interno del payload dell'*Issuer*.

Postcondizioni:

- La firma crittografica del *Key Binding* JWT è stata verificata mediante la chiave pubblica dell'*Holder*.
- Il parametro `nonce` contenuto nel KB-JWT corrisponde al *nonce* della sessione attiva.
- Il claim `aud` (*audience*) nel KB-JWT corrisponde al `clientIdValue` del Verifier.
- Il digest `sd_hash` presente nel KB-JWT coincide con l'hash calcolato sulla presentazione SD-JWT.

Scenario principale:

1. Il sistema estrae la chiave pubblica dell'*Holder* dal claim `cnf.jwk` contenuto nel payload dell'*Issuer* JWT.

2. Viene verificata la firma crittografica del KB-JWT, accertando contestualmente che il claim *audience* corrisponda al `clientIdValue` atteso del Verifier e che l'età del token non ecceda la soglia configurata.
3. Il Verifier confronta il valore di `nonce` presente nel payload del KB-JWT con il *nonce* registrato nella sessione corrente.
4. Viene calcolato il digest `sd_hash` della presentazione e confrontato con il corrispondente claim `sd_hash` dichiarato nel KB-JWT.

Scenario alternativo:

- Nel caso in cui l'*Issuer* JWT risulti privo del claim `cnf.jwk` necessario per la verifica dell'*Holder Binding*, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui la verifica della firma crittografica del KB-JWT non abbia esito positivo, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui il valore di `nonce` non coincida con quello memorizzato nella sessione, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui il valore calcolato di `sd_hash` non corrisponda a quello dichiarato nel KB-JWT, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui la *Credential Query* prescriva il vincolo di *Holder Binding* ma la presentazione risulti priva del KB-JWT, il sistema solleva un'eccezione di tipo `PreconditionFailedError`.

UC3.7: Validazione della completezza rispetto alla DCQL Query

Attori: Verifier.
Descrizione: Il Verifier accerta che il *VP Token* includa l'insieme completo delle credenziali richieste dalla *DCQL Query*, nel rispetto dei vincoli relativi alle credenziali obbligatorie. Viene inoltre verificato il vincolo di molteplicità (`multiple`), assicurando che il numero di presentazioni fornite per ciascuna query sia conforme a quanto stabilito.
Precondizioni:

- Il *VP Token* è stato analizzato sintatticamente con successo (rif. 2.2).
- La *DCQL Query* associata alla sessione è disponibile (rif. 2.2).

Postcondizioni:

- La conformità del *VP Token* rispetto ai vincoli di completezza e molteplicità della *DCQL Query* è stata verificata con esito positivo.

Scenario principale:

1. In assenza di `credentialSets`: il Verifier verifica che ciascun `credentialQueryId` specificato nella query di credenziale sia presente all'interno del *VP Token*.
2. In presenza di `credentialSets`: per ciascun insieme contrassegnato come obbligatorio, il sistema accerta che almeno un'opzione alternativa risulti integralmente soddisfatta nel *VP Token*.
3. Per ciascuna voce del *VP Token*, nel caso in cui la corrispondente query di credenziale non consenta presentazioni multiple, il sistema verifica che l'array contenga esattamente una singola presentazione.

Scenario alternativo:

- Nel caso in cui risultino assenti presentazioni per query di credenziale obbligatorie, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui nessuna delle combinazioni previste nei `credentialSets` risulti integralmente soddisfatta, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui una query non ammetta presentazioni multiple ma l'array associato contenga più di un elemento, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC3.8: Verifica crittografica della presentazione SD-JWT

Attori: Verifier.
Descrizione: Per ciascuna presentazione SD-JWT contenuta nel *VP Token*, il Verifier conduce una verifica crittografica esaustiva: esegue il *parsing* della struttura SD-JWT, risolve la chiave pubblica dell'*Issuer*, valida la firma crittografica dell'*Issuer* JWT e verifica l'integrità delle *Selective Disclosure*.
Precondizioni:

- La procedura di validazione DCQL è in corso di esecuzione (rif. 2.2).
- La presentazione fornita costituisce una stringa SD-JWT sintatticamente valida.

Postcondizioni:

- L'*Issuer* JWT è stato verificato sotto il profilo crittografico.
- Le *disclosure* presentate sono state validate e i claim selettivamente rivelati sono stati estratti.

Scenario principale:

1. Il Verifier scompone la presentazione SD-JWT nei suoi elementi costitutivi: *Issuer* JWT, array delle *disclosure* e, ove presente, *Key Binding* JWT.
2. Risoluzione della chiave pubblica dell'*Issuer*:
   - nel caso in cui siano state configurate delle *trusted authorities*, la chiave pubblica viene risolta mediante la verifica della *Trust Chain*;
   - in assenza di *trusted authorities*, la chiave pubblica viene estratta direttamente dall'header dell'*Issuer* JWT attraverso i parametri `jwk` o `x5c`.
3. La firma crittografica dell'*Issuer* JWT viene sottoposta a verifica.
4. Per ciascuna *disclosure* inclusa nella presentazione:
   - viene calcolato il digest crittografico impiegando l'algoritmo specificato nel payload;
   - il digest calcolato viene confrontato con l'insieme degli hash contenuti nel payload dell'*Issuer*;
   - la *disclosure* viene decodificata da Base64url e deserializzata come array JSON;
   - il claim rivelato viene integrato nella struttura dei claim divulgati.

Scenario alternativo:

- Nel caso in cui la stringa SD-JWT sia malformata, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui non sia possibile risolvere la chiave pubblica dell'*Issuer*, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui la verifica della firma dell'*Issuer* JWT non abbia esito positivo, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui il digest di una *disclosure* non figuri tra gli hash autorizzati nell'array `_sd`, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui l'algoritmo specificato nel parametro `_sd_alg` non sia supportato dal sistema, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC3.9: Validazione dei criteri delle credenziali (VCT e claim)

Attori: Verifier.
Descrizione: Il Verifier accerta che le credenziali presentate soddisfino i criteri specificati nella *DCQL Query*: il tipo di credenziale (*Verifiable Credential Type*, `vct`) deve risultare conforme ai valori attesi, e i claim richiesti devono essere presenti e assumere esclusivamente i valori consentiti.
Precondizioni:

- La presentazione SD-JWT è stata verificata crittograficamente con esito positivo (rif. 2.2).
- I claim selettivamente rivelati e il payload dell'*Issuer* sono disponibili.

Postcondizioni:

- Il tipo di credenziale (`vct`) è stato validato con successo.
- Tutti i claim richiesti dalla *DCQL Query* risultano presenti e conformi ai vincoli stabiliti.

Scenario principale:

1. Validazione del tipo di credenziale: il sistema accerta che il claim `vct` presente nel payload dell'*Issuer* sia compreso tra i valori definiti nella query di credenziale.
2. Validazione dei claim in assenza di `claimSets`: per ciascun claim richiesto, il sistema esamina la struttura unificata del payload percorrendo il cammino gerarchico specificato (*path*) e verifica la presenza e il valore dell'attributo associato.
3. Validazione dei claim in presenza di `claimSets`: il sistema itera sulle combinazioni alternative di `claimSets`, verificando per ciascun insieme che tutti i claim referenziati tramite identificativo risultino soddisfatti.

Scenario alternativo:

- Nel caso in cui l'*Issuer* JWT risulti privo del claim obbligatorio `vct`, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il valore di `vct` non figuri nell'array dei valori attesi (`vctValues`), il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui un claim obbligatorio non sia presente nel payload, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il valore di un claim non coincida con alcuno dei valori consentiti, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui nessuna delle alternative definite nei `claimSets` risulti soddisfatta, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC3.10: Creazione e persistenza del Response Record

Attori: Verifier.
Descrizione: A seguito del completamento con esito positivo di tutte le verifiche crittografiche e strutturali sulla presentazione, il Verifier genera un *Response Record*, lo persiste su Redis unitamente al *VP Token* validato e a un identificativo monouso (`responseCode`), e compone l'URI di reindirizzamento da restituire al Wallet nella risposta HTTP.
Precondizioni:

- Tutte le fasi di validazione sono state completate con esito positivo.
- La sessione si trova nello stato `response_received` (rif. 2.2).

Postcondizioni:

- Un oggetto `ResponseRecord` in stato `pending_fetch` è stato persistito su Redis, indicizzato mediante il `transactionId`.
- Il codice di risposta monouso `responseCode` è stato generato e associato al record.
- L'URI di reindirizzamento è stato restituito al Wallet.

Scenario principale:

1. Il componente `SessionService` genera un codice crittografico casuale `responseCode`.
2. Viene istanziato un oggetto `ResponseRecord` contenente i seguenti parametri: `vpToken`, `responseCode`, stato impostato a `pending_fetch` e `transactionId`.
3. Il record viene persistito su Redis con il valore di TTL configurato.

4. Il Verifier compone il parametro `redirect_uri` e lo restituisce al Wallet all'interno del payload della risposta HTTP con codice di stato `200 OK`.

UC4: Recupero della presentazione (fetch)

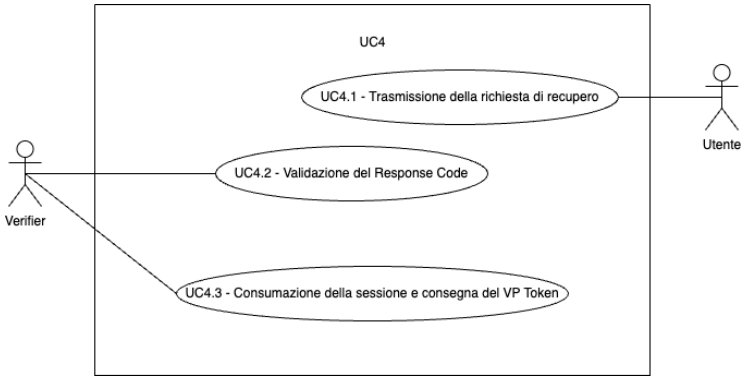


Figura 2.5: UC4 – Recupero della presentazione (fetch)

Attori: RP Frontend, Verifier.
Descrizione: L'interfaccia *frontend* del Relying Party, a seguito del reindirizzamento operato dal Wallet e recante il parametro `response_code`, trasmette una richiesta al Verifier finalizzata al recupero dei dati relativi alle credenziali verificate dell'Utente. Il Verifier accerta la validità del `responseCode`, restituisce il *VP Token* e provvede alla rimozione contestuale della sessione e del *Response Record* dall'istanza Redis, garantendo la natura monouso dell'operazione di recupero.
Precondizioni:

- Il *frontend* del Relying Party è in possesso del parametro `sessionId` (rif. 2.2) e del parametro `responseCode` (rif. 2.2).
- La sessione associata al `sessionId` risulta presente nell'istanza Redis.
- Il `ResponseRecord` associato è presente su Redis e non risulta precedentemente consumato, trovandosi nello stato `pending_fetch`.
- Il parametro `responseCode` corrisponde a quello generato e persistito dal sistema.

Postcondizioni:

- Il *frontend* del Relying Party ha ricevuto il *VP Token* contenente le credenziali verificate dell'Utente.
- Lo stato della sessione è stato aggiornato a `completed`.
- La sessione e il `ResponseRecord` sono stati eliminati dall'istanza Redis.

Scenario principale:

1. Il Wallet effettua il reindirizzamento verso il *frontend* del Relying Party, trasmettendo il parametro `response_code` come frammento dell'URI.
2. Il *frontend* trasmette una richiesta HTTP `POST` all'endpoint, includendo nel payload i parametri `sessionId` e `responseCode`.
3. Il Verifier recupera la sessione e il `ResponseRecord` dall'istanza Redis.
4. Il Verifier accerta che il parametro `responseCode` fornito corrisponda a quello registrato nel `ResponseRecord`.
5. Il Verifier verifica che il `ResponseRecord` non sia stato precedentemente consumato.
6. Lo stato della sessione viene aggiornato a `completed`.

7. La sessione e il `ResponseRecord` vengono rimossi dall'istanza Redis.
8. Il Verifier restituisce il `vpToken` al *frontend* all'interno di una risposta HTTP con codice di stato `200 OK`.

Scenario alternativo:

- Nel caso in cui la sessione non risulti presente nell'istanza Redis, il sistema solleva un'eccezione di tipo `NotFoundError`.
- Nel caso in cui il `ResponseRecord` non sia reperibile su Redis, il sistema solleva un'eccezione di tipo `NotFoundError`.
- Nel caso in cui il parametro `responseCode` fornito non coincida con quello persistito nel `ResponseRecord`, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il `ResponseRecord` risulti già nello stato `consumed`, il sistema solleva un'eccezione di tipo `PreconditionFailedError`.

UC4.1: Trasmissione della richiesta di recupero

Attori: Utente (tramite RP Frontend), Verifier.
Descrizione: L'Utente, per il tramite dell'interfaccia *frontend* del Relying Party, inoltra una richiesta HTTP `POST` all'endpoint del Verifier al fine di acquisire le credenziali verificate. La richiesta comprende il parametro `sessionId` (rif. 2.2) e il parametro `responseCode` (rif. 2.2).
Precondizioni:

- L'Utente è in possesso del parametro `sessionId` generato in fase di inizializzazione della richiesta di presentazione (rif. 2.2).
- Il Wallet ha completato la procedura di presentazione (rif. 2.2) e il *frontend* ha acquisito il parametro `responseCode` dall'URI di reindirizzamento.
- L'endpoint di *fetch* del Verifier è raggiungibile e operativo.

Postcondizioni:

- La richiesta HTTP `POST` è stata ricevuta dal Verifier unitamente al payload contenente i parametri `sessionId` e `responseCode`.

Scenario principale:

1. Il *frontend* del Relying Party estrae il parametro `responseCode` dal frammento dell'URI di reindirizzamento.
2. Viene composto il corpo della richiesta HTTP `POST`, includendo i campi `sessionId` e `responseCode`.
3. Il *frontend* inoltra la richiesta all'endpoint di *fetch* del Verifier.

Scenario alternativo:

- Nel caso in cui l'URI di reindirizzamento risulti privo del parametro `response_code`, il *frontend* segnala la condizione di errore all'Utente e interrompe il flusso operativo senza procedere alla trasmissione della richiesta di *fetch*.

UC4.2: Validazione del Response Code

Attori: Verifier.
Descrizione: Il Verifier accerta che il codice monouso `responseCode` trasmesso dal *frontend* corrisponda esattamente a quello registrato nel *Response Record* associato alla sessione e che tale record non sia stato precedentemente consumato, prevenendo eventuali tentativi di riutilizzo o di enumerazione dei codici di risposta.
Precondizioni:

- Il Verifier ha ricevuto la richiesta di *fetch* contenente i parametri `sessionId` e `responseCode` (rif. 2.2).

- La sessione e il corrispondente `ResponseRecord` risultano presenti nell'istanza Redis.

Postcondizioni:

- Il parametro `responseCode` è stato validato con esito positivo e il record associato risulta integro e non precedentemente consumato.

Scenario principale:

1. Il Verifier recupera la sessione dall'istanza Redis mediante il parametro `sessionId`.
2. Il Verifier recupera il `ResponseRecord` da Redis tramite il `transactionId` associato alla sessione.
3. Il parametro `responseCode` fornito nella richiesta viene confrontato con quello custodito nel `ResponseRecord`.
4. Il Verifier accerta che lo stato del `ResponseRecord` corrisponda a `pending_fetch`.

Scenario alternativo:

- Nel caso in cui la sessione non risulti presente nell'istanza Redis, il sistema solleva un'eccezione di tipo `NotFoundError`.
- Nel caso in cui il `ResponseRecord` non sia reperibile su Redis, il sistema solleva un'eccezione di tipo `NotFoundError`.
- Nel caso in cui il parametro `responseCode` non corrisponda a quello registrato nel record, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il `ResponseRecord` risulti già nello stato `consumed`, il sistema solleva un'eccezione di tipo `PreconditionFailedError`.

UC4.3: Consumazione della sessione e consegna del VP Token

Attori: Verifier.
Descrizione: A seguito della validazione del codice di risposta, il Verifier aggiorna lo stato della sessione a `completed`, provvede alla rimozione della sessione e del *Response Record* dall'istanza Redis al fine di precluderne qualsivoglia riutilizzo, e restituisce il *VP Token* validato al *frontend* del Relying Party.
Precondizioni:

- Il parametro `responseCode` è stato validato con esito positivo (rif. 2.2).

Postcondizioni:

- Lo stato della sessione è stato aggiornato a `completed`.
- La sessione e il `ResponseRecord` sono stati rimossi dall'istanza Redis.
- Il *frontend* del Relying Party ha ricevuto il *VP Token* contenente le credenziali verificate dell'Utente.

Scenario principale:

1. Il Verifier aggiorna lo stato della sessione a `completed`.
2. Il Verifier elimina la sessione e il `ResponseRecord` dall'istanza Redis.
3. Il Verifier restituisce l'oggetto `vpToken` al *frontend* del Relying Party.

UC5: Gestione del ciclo di vita del client

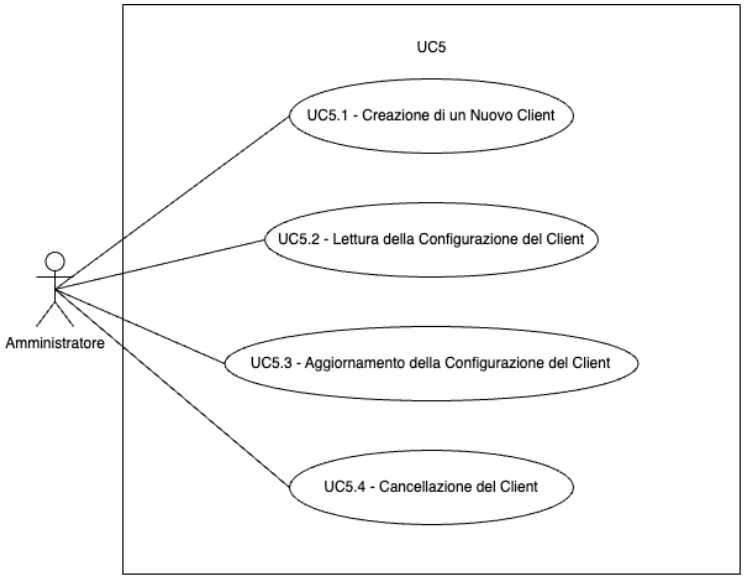


Figura 2.6: UC5 – Gestione del ciclo di vita del client

Attori: Amministratore, Verifier.
Descrizione: L'Amministratore gestisce la configurazione dei client (Verifier) nel sistema attraverso le API SCIM. Tale operazione comprende le funzionalità di creazione, lettura, aggiornamento e cancellazione (CRUD) delle configurazioni dei client, che definiscono parametri fondamentali quali i formati *Verifiable Presentation* supportati, gli URI di reindirizzamento, il set di chiavi crittografiche JWKS e gli algoritmi crittografici abilitati per la firma e la cifratura.
Precondizioni:

- L'Amministratore dispone di credenziali di accesso autenticate e autorizzate per le API SCIM del sistema.
- L'istanza del database MongoDB risulta raggiungibile e operativa.

Postcondizioni:

- Un nuovo client è stato registrato nel database con la configurazione specificata.
- L'Amministratore ha acquisito la configurazione del client richiesto ovvero l'elenco dei client associati al dominio.
- La configurazione del client preesistente è stata modificata conformemente ai parametri forniti.
- Il record del client è stato rimosso in modo permanente dal sistema.

Scenario principale:

1. **Creazione:** l'Amministratore trasmette una richiesta HTTP `POST` all'endpoint, includendo nel payload la configurazione del client con i campi obbligatori `redirectUris` e `vpFormatsSupported`. Il Verifier valida la conformità dei campi, persiste il record nell'istanza MongoDB e restituisce la risorsa creata.
2. **Lettura:** l'Amministratore inoltra una richiesta HTTP `GET` all'endpoint al fine di acquisire la configurazione di un client specifico, oppure di ottenere l'elenco completo dei client registrati nel dominio.
3. **Aggiornamento:** l'Amministratore trasmette una richiesta HTTP `PUT` o `PATCH` all'endpoint, includendo i parametri oggetto di modifica; il sistema valida le variazioni e aggiorna il record corrispondente su MongoDB.

4. **Cancellazione:** l'Amministratore inoltra una richiesta HTTP `DELETE` all'endpoint al fine di rimuovere definitivamente il client dal sistema.

Scenario alternativo:

- Nel caso in cui i campi obbligatori `redirectUris` o `vpFormatsSupported` risultino assenti o privi di valore in fase di creazione, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui un parametro che prescrive una struttura ad array non sia fornito nel formato atteso, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il client individuato mediante l'identificativo non risulti presente all'interno del dominio indicato, il sistema solleva un'eccezione di tipo `NotFoundError`.

UC5.1: Creazione di un nuovo client

Attori: Amministratore, Verifier.
Descrizione: L'Amministratore registra una nuova configurazione di client (Verifier) all'interno del sistema, trasmettendo i parametri obbligatori e opzionali attraverso le API SCIM.
Precondizioni:

- L'Amministratore dispone di credenziali di accesso autenticate e autorizzate per le API SCIM del sistema.
- L'istanza del database MongoDB risulta raggiungibile e operativa.

Postcondizioni:

- Un nuovo record relativo al client è stato validato e persistito nell'istanza MongoDB con la configurazione specificata.

Scenario principale:

1. L'Amministratore trasmette una richiesta HTTP `POST` all'endpoint, il cui corpo include obbligatoriamente i campi `redirectUris` (array di stringhe) e `vpFormatsSupported` (con specifica del formato `dcSdJwt`), unitamente ad eventuali parametri di configurazione crittografica aggiuntivi.
2. Il Verifier esegue la validazione della richiesta: accerta la presenza dei campi obbligatori, verifica la conformità al formato array per tutti i campi prescritti e valida la corretta composizione del parametro `vpFormatsSupported`.
3. Il Verifier persiste il record del client nell'istanza MongoDB.

Scenario alternativo:

- Nel caso in cui il parametro obbligatorio `redirectUris` risulti assente, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il parametro obbligatorio `vpFormatsSupported` risulti assente, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui un parametro che prescrive una struttura ad array non sia fornito nel formato corretto, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC5.2: Lettura della configurazione del client

Attori: Amministratore, Verifier.
Descrizione: L'Amministratore interroga il sistema per recuperare la configurazione completa di uno specifico client ovvero l'elenco di tutti i client censiti per un determinato dominio.
Precondizioni:

- L'Amministratore dispone di credenziali di accesso autenticate e autorizzate per le API SCIM del sistema.
- L'istanza del database MongoDB risulta raggiungibile e operativa.

Postcondizioni:

- La configurazione del client richiesto ovvero l'elenco dei client del dominio è stato restituito all'Amministratore.

Scenario principale:

1. L'Amministratore trasmette una richiesta HTTP `GET` all'endpoint specificando l'identificativo del client; il Verifier recupera il record dall'istanza MongoDB e restituisce l'oggetto di configurazione completo all'interno di una risposta HTTP `200 OK`.
2. L'Amministratore trasmette una richiesta HTTP `GET` all'endpoint al fine di ottenere la lista di tutti i client registrati nel dominio; il Verifier interroga MongoDB e restituisce la collezione dei client associati.

Scenario alternativo:

- Nel caso in cui il client specificato non risulti presente all'interno del dominio indicato, il sistema solleva un'eccezione di tipo `NotFoundError`.

UC5.3: Aggiornamento della configurazione del client

Attori: Amministratore, Verifier.
Descrizione: L'Amministratore apporta modifiche parziali o procede alla sostituzione integrale della configurazione di un client precedentemente registrato nel sistema.
Precondizioni:

- L'Amministratore dispone di credenziali di accesso autenticate e autorizzate per le API SCIM del sistema.
- Il client oggetto di modifica risulta presente nell'istanza MongoDB per il dominio specificato.

Postcondizioni:

- La configurazione del client è stata aggiornata su MongoDB con i nuovi parametri validati.

Scenario principale:

1. L'Amministratore trasmette una richiesta HTTP `PUT`, per la sostituzione integrale della configurazione, ovvero HTTP `PATCH`, per l'aggiornamento parziale dei campi, all'endpoint designato.
2. Il Verifier valida la conformità sintattica e strutturale dei parametri forniti.
3. Il Verifier persiste le modifiche nell'istanza MongoDB e restituisce la risorsa aggiornata con codice di stato HTTP `200 OK`.

Scenario alternativo:

- Nel caso in cui il client specificato non risulti presente all'interno del dominio, il sistema solleva un'eccezione di tipo `NotFoundError`.
- Nel caso in cui uno o più parametri oggetto di aggiornamento non rispettino i vincoli di validazione sintattica o strutturale, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC5.4: Cancellazione del client

Attori: Amministratore, Verifier.
Descrizione: L'Amministratore dispone la rimozione permanente della configurazione di un client dal sistema.
Precondizioni:

- L'Amministratore dispone di credenziali di accesso autenticate e autorizzate per le API SCIM del sistema.
- Il client da eliminare risulta presente nell'istanza MongoDB per il dominio specificato.

Postcondizioni:

- Il record relativo al client è stato eliminato in modo definitivo dall'istanza MongoDB.

Scenario principale:

1. L'Amministratore trasmette una richiesta HTTP `DELETE` all'endpoint designato, specificando l'identificativo del client oggetto di cancellazione.
2. Il Verifier accerta la presenza del record nell'istanza MongoDB ed esegue l'eliminazione definitiva del client.
3. Il Verifier restituisce una risposta di conferma attestante l'avvenuta cancellazione.

Scenario alternativo:

- Nel caso in cui il client specificato non risulti presente all'interno del dominio indicato, il sistema solleva un'eccezione di tipo `NotFoundError`.

UC6: Validazione della Trust Chain

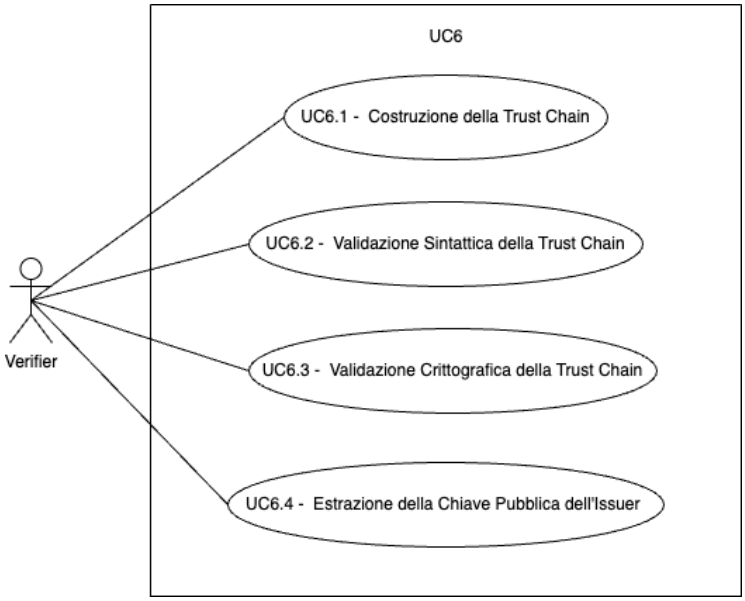


Figura 2.7: UC6 – Validazione della Trust Chain

Attori: Verifier.
Descrizione: Qualora il flusso di autenticazione o presentazione impieghi il prefisso `openid_federation`, il Verifier deve costruire e validare una *Trust Chain* gerarchica a partire dall'entità foglia (*Leaf Entity*) fino a un'ancora di fiducia (*Trust Anchor*) preconfigurata. Tale procedura certifica l'affiliazione dell'entità a una federazione di fiducia, accertando la catena ininterrotta delle firme crittografiche, la validità temporale dei singoli *statement* e la coerenza delle relazioni tra soggetti (`sub`) ed emittenti (`iss`) lungo l'intera gerarchia.
Precondizioni:

- Le autorità fidate (*Trusted Authorities*) con schema `openid_federation` sono specificate nella query DCQL (`dcqlQuery`).
- Il *Trust Anchor* è raggiungibile via rete e il corrispondente set di chiavi crittografiche JWKS è stato preventivamente memorizzato nell'istanza Redis.
- Gli endpoint delle entità costituenti la catena risultano raggiungibili e operativi.

Postcondizioni:

- La *Trust Chain* è stata costruita con esito positivo dalla *Leaf Entity* al *Trust Anchor*.

- Tutte le firme crittografiche degli *statement* appartenenti alla catena sono state verificate con successo.
- Tutti gli *statement* della catena risultano temporalmente validi, con il claim `iat` collocato nel passato e il claim `exp` nel futuro rispetto all'istante corrente.
- La chiave pubblica dell'Issuer o dell'entità target è stata estratta dai metadati della *Leaf Entity*.

Scenario principale:

1. Il Verifier avvia la procedura di risoluzione della catena acquisendo l'*Entity Configuration* della *Leaf Entity* dal relativo endpoint.
2. Il Verifier accerta che i claim `iss` e `sub` della configurazione coincidano con il parametro `entityId` atteso.
3. Il Verifier risale ricorsivamente la gerarchia federativa seguendo i riferimenti dichiarati nel claim `authority_hints`, recuperando per ciascuna entità intermedia:
   - l'*Entity Configuration* dell'intermediario dal rispettivo endpoint;
   - il *Subordinate Statement* emesso dal superiore.
4. La ricorsione termina non appena viene raggiunta un'entità identificata come *Trust Anchor* autorizzato.
5. Il Verifier esegue la validazione sintattica di ciascun documento della catena, verificando la presenza dei claim obbligatori (`iss`, `sub`, `iat`, `exp`) e la loro coerenza temporale.
6. Il Verifier esegue la validazione crittografica della catena.
7. Viene accertata la coerenza dell'intera catena, assicurando che il claim `sub` di ciascun *Subordinate Statement* corrisponda al soggetto dell'entità sottostante.
8. La chiave pubblica dell'Issuer viene estratta dai metadati della *Leaf Entity*.

Scenario alternativo:

- Nel caso in cui non sia configurata alcuna *Trusted Authority* con prefisso `openid_federation`, il sistema solleva un'eccezione di tipo `ForbiddenError`.
- Nel caso in cui la risoluzione ricorsiva non consenta di raggiungere alcun *Trust Anchor* valido, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui venga rilevata una dipendenza ciclica, il ramo di risoluzione corrente viene interrotto mediante *backtracking*.
- Nel caso in cui il JWKS del *Trust Anchor* non risulti presente nell'istanza Redis, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il claim di scadenza `exp` di un qualunque *statement* risulti collocato nel passato, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui la verifica della firma crittografica non abbia esito positivo su uno degli elementi della catena, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui si riscontri una discrepanza tra i claim `iss` e `sub` lungo la catena, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC6.1: Costruzione della Trust Chain

Attori: Verifier.
Descrizione: Il Verifier costruisce la *Trust Chain* gerarchica a partire dalla configurazione dell'entità foglia (*Leaf Entity*), risalendo ricorsivamente i riferimenti espressi nel claim `authority_hints` sino al raggiungimento di un'ancora di fiducia (*Trust Anchor*) preconfigurata. Durante la traversata della gerarchia vengono acquisite le *Entity Configuration* e i corrispondenti *Subordinate Statement* di ciascuna entità intermedia.
Precondizioni:

- L'identificativo `entityId` della *Leaf Entity* è noto al sistema.
- Almeno un *Trust Anchor* risulta configurato nell'insieme delle autorità fidate (`trustedAuthorities`).

- Gli endpoint standard `.well-known/openid-federation` delle entità federate risultano raggiungibili via HTTP.

Postcondizioni:

- È stata costruita una sequenza ordinata di token JWT rappresentante la catena di fiducia.

Scenario principale:

1. Il Verifier acquisisce l'*Entity Configuration* della *Leaf Entity* dal rispettivo endpoint.
2. Viene accertata la coincidenza dei claim `iss` e `sub` con il valore di `entityId` atteso.
3. Nel caso in cui la *Leaf Entity* coincida con un *Trust Anchor*, il Verifier restituisce la catena unitaria.
4. In caso contrario, il Verifier itera sui riferimenti contenuti nel claim `authority_hints`:
   - acquisisce l'*Entity Configuration* di ciascuna entità intermedia (*hint*);
   - verifica la corrispondenza dei claim `iss` e `sub` dell'intermediario;
   - acquisisce il *Subordinate Statement* interrogando l'endpoint dell'intermediario con i parametri `sub` (riferito all'entità sottostante) e `iss` (riferito all'intermediario);
   - prosegue ricorsivamente la risalita della catena a partire dall'intermediario.
5. Un meccanismo di prevenzione dei cicli tiene traccia delle entità precedentemente visitate (`visitedEntities`).

Scenario alternativo:

- Nel caso in cui un'*Entity Configuration* presenti una discrepanza tra i claim `iss`, `sub` e il corrispondente `entityId`, il relativo ramo di risoluzione viene scartato.
- Nel caso in cui un'entità intermedia non esponga l'endpoint richiesto, il relativo ramo federativo viene scartato.
- Nel caso in cui venga rilevata una visita ciclica a un'entità precedentemente elaborata, il ramo corrente viene interrotto.
- Nel caso in cui nessun percorso consenta di raggiungere un *Trust Anchor* valido, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC6.2: Validazione sintattica della Trust Chain

Attori: Verifier.
Descrizione: Il Verifier accerta la conformità sintattica di ogni *Entity Statement* e di ogni *Entity Configuration* appartenente alla catena, verificando la presenza dei claim obbligatori e il rispetto dei vincoli temporali.
Precondizioni:

- La *Trust Chain* è stata costruita con successo (rif. 2.2).

Postcondizioni:

- L'insieme dei documenti della catena contiene i claim prescritti e risulta temporalmente valido.

Scenario principale:

1. Per ciascun token JWT costituente la catena, il Verifier decodifica l'*header* e il *payload*.
2. Viene accertata la presenza dei claim obbligatori: `iss`, `sub`, `iat` ed `exp`.
3. Viene verificato che il claim `iat` (*issued at*) non risulti collocato nel futuro rispetto all'istante corrente.
4. Viene accertato che il claim `exp` (*expiration time*) non risulti scaduto rispetto al tempo corrente.

Scenario alternativo:

- Nel caso in cui risulti assente uno dei claim obbligatori, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il claim `iat` risulti posteriore all'istante corrente, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il claim `exp` risulti anteriore all'istante corrente, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC6.3: Validazione crittografica della Trust Chain

Attori: Verifier.
Descrizione: Il Verifier accerta crittograficamente l'integrità e l'autenticità di ogni firma presente nella catena: la firma del *Trust Anchor* è verificata tramite il relativo JWKS *out-of-band* (persistito su Redis), le firme dei *Subordinate Statement* intermedi sono verificate mediante i JWKS degli enti sovraordinati, e l'*Entity Configuration* della *Leaf Entity* è validata tramite auto-firma.
Precondizioni:

- La sequenza dei documenti della catena è stata decodificata e validata sintatticamente (rif. 2.2).
- Il set di chiavi JWKS del *Trust Anchor* risulta preventivamente caricato nell'istanza Redis.

Postcondizioni:

- Tutte le firme crittografiche della catena sono state verificate con esito positivo.
- La coerenza relazionale dei claim (`iss`/`sub`) tra elementi adiacenti è stata confermata.

Scenario principale:

1. Il Verifier recupera il JWKS del *Trust Anchor* dall'istanza Redis tramite il claim `iss`; accerta che `iss` coincida con `sub` e verifica la firma crittografica mediante la chiave pubblica *out-of-band*.
2. Per ciascun documento intermedio, il Verifier accerta che il claim `iss` dell'elemento corrente coincida con il claim `sub` dell'elemento sovraordinato, procedendo alla verifica della firma crittografica tramite il JWKS esposto nel *payload* dell'elemento sovraordinato.
3. Il Verifier accerta che `iss` coincida con `sub` nell'*Entity Configuration* della *Leaf Entity*, ne verifica l'auto-firma mediante le chiavi JWKS dell'entità medesima e accerta che il claim `sub` del primo *Subordinate Statement* coincida con il soggetto della *Leaf Entity*.

Scenario alternativo:

- Nel caso in cui il JWKS del *Trust Anchor* non risulti presente nell'istanza Redis, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui il JWKS del *Trust Anchor* non risulti sintatticamente decodificabile, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui si riscontri una discrepanza tra i claim `iss` e `sub` nel *Trust Anchor*, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui la catena risulti interrotta a causa di una non corrispondenza tra claim `iss` e `sub` adiacenti, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui fallisca la verifica della firma crittografica su uno qualunque degli *statement*, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui un *Entity Statement* risulti privo del set di chiavi JWKS richiesto, il sistema solleva un'eccezione di tipo `InvalidValueError`.

UC6.4: Estrazione della chiave pubblica dell'Issuer

Attori: Verifier.

Descrizione: Al completamento della validazione della *Trust Chain*, il Verifier estrae la chiave pubblica dell'Issuer dai metadati contenuti nell'*Entity Configuration* della *Leaf Entity*, rendendola disponibile per la successiva verifica crittografica dell'Issuer JWT (rif. 2.2).
Precondizioni:

- La *Trust Chain* è stata validata crittograficamente con esito positivo (rif. 2.2).

Postcondizioni:

- La chiave pubblica dell'Issuer è stata estratta e importata come oggetto `CryptoKey`.

Scenario principale:

1. Il Verifier accede alla sezione `metadata` dell'*Entity Configuration* della *Leaf Entity*.
2. Il sistema ricerca il set di chiavi JWKS all'interno dei ruoli metadati definiti, adottando una politica di risoluzione a cascata.
3. Qualora l'*header* del token indichi un identificativo di chiave (`kid`), viene selezionata la chiave corrispondente dal set JWKS; in assenza di indicazione esplicita, viene impiegata la prima chiave disponibile.
4. Il Verifier provvede all'importazione della chiave pubblica mediante la libreria crittografica (invocando `jose.importJWK()`), assicurando la rimozione di eventuali parametri riservati.

Scenario alternativo:

- Nel caso in cui la sezione `metadata` risulti assente dall'*Entity Configuration*, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui non sia reperibile alcun set JWKS valido tra i ruoli censiti nei metadati, il sistema solleva un'eccezione di tipo `InvalidValueError`.
- Nel caso in cui l'operazione di importazione crittografica della chiave pubblica non abbia esito positivo, il sistema solleva un'eccezione di tipo `InvalidValueError`.

2.3 Tracciamento dei requisiti

A seguito dell'analisi di dettaglio e della formalizzazione dei casi d'uso, è stato possibile definire il quadro completo dei requisiti del sistema. Tale catalogazione risulta essenziale per garantire che ciascuna funzionalità implementata risponda puntualmente agli obiettivi prefissati dall'organizzazione proponente e per supportare in modo sistematico le successive fasi di progettazione, verifica e collaudo.
Al fine di classificare i requisiti in modo univoco, rigoroso e strutturato, è stata adottata la seguente notazione alfanumerica:

`R[Tipologia][Importanza]-[Numero progressivo]`

Tipologia del requisito:

- **F** (*Funzionale*): individua le funzionalità, i comportamenti operativi e i servizi che il sistema deve erogare agli attori coinvolti;
- **Q** (*Qualitativo*): definisce gli standard di qualità del software, con particolare riferimento a manutenibilità, affidabilità, modularità e leggibilità del codice sorgente;
- **V** (*Di Vincolo*): prescrive le restrizioni tecnologiche, architetturali o normative derivanti dal contesto operativo e dall'infrastruttura preesistente.

Importanza del requisito:

- **O** (*Obbligatorio*): requisito essenziale e vincolante, la cui mancata soddisfazione pregiudica l'accettazione e il corretto funzionamento dell'intero sistema;
- **D** (*Desiderabile*): requisito non strettamente vincolante ma caratterizzato da un elevato valore aggiunto in termini di completezza ed estendibilità della piattaforma;

- **F** (*Facoltativo*): requisito secondario, la cui implementazione è subordinata al completamento dei requisiti prioritari e alla disponibilità di tempo residuo.

Nelle sezioni seguenti vengono presentati i requisiti individuati, evidenziandone la classificazione e la correlazione con i casi d'uso definiti in fase di analisi.

2.3.1 Requisiti funzionali

Tabella 2.1: Requisiti funzionali

Requisito	Descrizione	Use Case
RFO-1	Il sistema deve permettere l'inizializzazione di una richiesta di presentazione di credenziali verificabili.	UC1
RFO-2	Il sistema deve recuperare la configurazione del client dal database.	UC1.1
RFO-3	Il sistema deve validare il prefisso del Client ID e determinarne le proprietà di firma.	UC1.2
RFO-4	Il sistema deve creare una sessione con identificativi univoci e URL-safe.	UC1.4
RFO-5	Il sistema deve costruire l'URL di autorizzazione in base al tipo di prefisso del client.	UC1.5
RFO-6	Il sistema deve generare il Request Object JWT su richiesta del Wallet.	UC2
RFO-7	Il sistema deve verificare che il Request URI non sia già stato consumato.	UC2.1
RFO-8	Il sistema deve analizzare i `wallet_metadata` per negoziare gli algoritmi crittografici comuni.	UC2.2
RFO-9	Il sistema deve assemblare il payload del Request Object con i parametri del protocollo OID4VP.	UC2.4
RFO-10	Il sistema deve firmare il Request Object con la chiave privata del client.	UC2.6
RFO-11	Il sistema deve impedire il riutilizzo del Request URI aggiornando la sessione dopo la generazione.	UC2.7
RFO-12	Il Wallet deve poter trasmettere le *Verifiable Presentation* al Verifier.	UC3.1
RFO-13	Il sistema deve estrarre i parametri dalla risposta del Wallet, supportando i formati `direct_post` e `direct_post.jwt`.	UC3.2
RFO-14	Il sistema deve recuperare la sessione e aggiornarla atomicamente allo stato `response_received`.	UC3.4
RFO-15	Il sistema deve parsare e validare la struttura del *VP Token*.	UC3.5
RFO-16	Il sistema deve validare la completezza del *VP Token* rispetto alla query DCQL.	UC3.7
RFO-17	Il sistema deve verificare crittograficamente ogni presentazione SD-JWT nel *VP Token*.	UC3.8
RFO-18	Il sistema deve validare il tipo di credenziale e i claim richiesti dalla query DCQL.	UC3.9

Continua nella pagina successiva...

Tabella 2.1 – Continua dalla pagina precedente

Requisito	Descrizione	Use Case
RFO-19	Il sistema deve generare un *Response Record* con `responseCode` monouso e restituire un *redirect URI* al Wallet.	UC3.10
RFO-20	L'Utente deve poter recuperare le credenziali verificate tramite `sessionId` e `responseCode`.	UC4.1
RFO-21	Il sistema deve validare il `responseCode` e verificare che non sia già stato consumato.	UC4.2
RFO-22	Il sistema deve consumare la sessione, eliminare i dati da Redis e restituire il *VP Token* all'Utente.	UC4.3
RFO-23	Il sistema deve permettere la creazione di un nuovo client.	UC5.1
RFO-24	Il sistema deve permettere la lettura della configurazione di un client.	UC5.2
RFO-25	Il sistema deve permettere l'aggiornamento della configurazione di un client.	UC5.3
RFO-26	Il sistema deve permettere la cancellazione di un client.	UC5.4
RFO-27	Il sistema deve supportare sei prefissi di Client ID conformi al protocollo OID4VP.	UC1.2
RFO-28	Il sistema deve gestire gli errori del Wallet aggiornando la sessione a `failed` e restituendo un *redirect URI* di errore.	UC3
RFD-1	Il sistema deve generare chiavi effimere JWE e salvarle su Redis per la cifratura della risposta.	UC2.3
RFD-2	Il sistema deve decifrare la risposta JWE del Wallet e distruggere la chiave effimera dopo l'uso.	UC3.3
RFD-3	Il sistema deve generare un *Entity Configuration* JWT per il protocollo OpenID Federation.	UC2.5
RFD-4	Il sistema deve validare la *Trust Chain* dell'OpenID Provider durante l'inizializzazione.	UC1.3
RFD-5	Il sistema deve costruire la *Trust Chain* tramite risoluzione ricorsiva degli *Entity Statement*.	UC6.1
RFD-6	Il sistema deve validare sintatticamente ogni *Entity Statement* nella *Trust Chain*.	UC6.2
RFD-7	Il sistema deve validare crittograficamente ogni firma nella *Trust Chain*.	UC6.3
RFD-8	Il sistema deve estrarre la chiave pubblica dell'Issuer dalla *Leaf Entity Configuration*.	UC6.4
RFD-9	Il sistema deve validare l'*Holder Binding* verificando il Key Binding JWT.	UC3.6
RFD-10	Il sistema deve verificare la corrispondenza tra chiave privata e certificato *leaf* per i prefissi X.509.	UC2.6
RFF-1	Il sistema deve supportare il fallback dell'algoritmo `enc` ad `A128GCM` in fase di negoziazione.	UC2.2

2.3.2 Requisiti qualitativi

Tabella 2.2: Requisiti qualitativi

Requisito	Descrizione	Use Case
RQO-1	Il sistema deve restituire messaggi di errore specifici per ogni condizione di errore.	–
RQO-2	Il sistema deve adottare una gestione centralizzata degli errori tramite middleware con mappatura verso i codici HTTP appropriati.	–
RQO-3	Il sistema deve registrare nei log ogni fase significativa del flusso di presentazione.	–
RQO-4	Il sistema deve convertire automaticamente tra le convenzioni *camelCase* e *snake_case* per i payload del protocollo.	–
RQD-1	Il codice deve essere organizzato secondo un'architettura a strati per garantire separazione delle responsabilità.	–
RQD-2	Il sistema deve utilizzare valori di default sicuri per i parametri configurabili.	–

2.3.3 Requisiti di vincolo

Tabella 2.3: Requisiti di vincolo

Requisito	Descrizione	Use Case
RVO-1	Il sistema deve essere implementato in TypeScript con runtime Node.js.	–
RVO-2	Il sistema deve utilizzare Redis per la gestione delle sessioni, delle chiavi effimere e dei *Response Record*.	–
RVO-3	Il sistema deve utilizzare MongoDB per la persistenza delle configurazioni dei client.	–
RVO-4	Il sistema deve utilizzare la libreria `jose` per le operazioni crittografiche JWT/JWE/JWS.	–
RVO-5	Il sistema deve esporre le API REST tramite il framework Express.js.	–
RVO-6	Il sistema deve implementare il protocollo OID4VP per la verifica di credenziali verificabili.	–
RVO-7	Il sistema deve supportare il formato SD-JWT come formato di credenziale verificabile.	–
RVO-8	Il sistema deve implementare il protocollo DCQL per la definizione dei requisiti di presentazione.	–
RVO-9	Il sistema deve utilizzare l'operazione atomica `GETDEL` di Redis per la distruzione delle chiavi effimere.	–
RVD-1	Il sistema deve supportare il protocollo OpenID Federation per la gestione della fiducia.	–
RVD-2	Il sistema deve supportare la containerizzazione tramite Docker.	–

2.3.4 Riepilogo quantitativo

Tabella 2.4: Riepilogo quantitativo dei requisiti

Tipologia	**Obbligatorio (O)**	**Desiderabile (D)**	**Facoltativo (F)**	**Totale**
Funzionali (RF)	28	10	1	39
Qualitativi (RQ)	4	2	0	6
Di Vincolo (RV)	9	2	0	11
Totale	**41**	**14**	**1**	**56**

Capitolo 3

Progettazione

In questo capitolo descriviamo la fase di progettazione del sistema, che segue l'Analisi dei Requisiti del Capitolo 2. Spieghiamo le scelte architetturali, la stratificazione del sistema, l'organizzazione della codebase, la modellazione dei dati di dominio, la strategia di persistenza e l'infrastruttura di sicurezza crittografica.

3.1 Architettura generale

I principi guida che hanno orientato ogni scelta architetturale sono:

- **Conformità agli standard:** le interfacce e i flussi del sistema sono progettati nel rispetto delle specifiche OID4VP$_{\text{G}}$, OpenID Federation 1.0 e SD-JWT$_{\text{G}}$, garantendo l'aderenza ai profili di interoperabilità definiti dall'ecosistema EUDI Wallet;
- **Sicurezza by design:** i controlli crittografici e le verifiche di integrità sono integrati nativamente in ogni fase del flusso, anziché essere aggiunti a posteriori;
- **Multi-tenancy$_{\text{G}}$:** l'architettura consente a molteplici organizzazioni di operare sulla medesima istanza del servizio, con isolamento logico dei dati e delle configurazioni;
- **Statelessness del server:** nessuno stato di sessione è mantenuto nella memoria volatile dei processi applicativi; ogni informazione transazionale è centralizzata su Redis;
- **Tolleranza ai guasti:** i meccanismi di scadenza automatica (TTL$_{\text{G}}$) e le transizioni di stato atomiche garantiscono la coerenza del sistema anche in presenza di interruzioni impreviste;
- **Interoperabilità:** il sistema è progettato per interagire con Digital Wallet$_{\text{G}}$ e Issuer eterogenei, supportando molteplici schemi di identificazione del client e suite crittografiche negoziabili.

Il sistema adotta un'architettura a microservizio distribuito con separazione netta tra i livelli (riassunta in Figura 3.1). Non essendo un'applicazione client-side, il progetto non contiene un livello di interfaccia utente proprio: il ruolo di *client* è svolto da componenti esterni (Frontend RP e Wallet dell'utente).

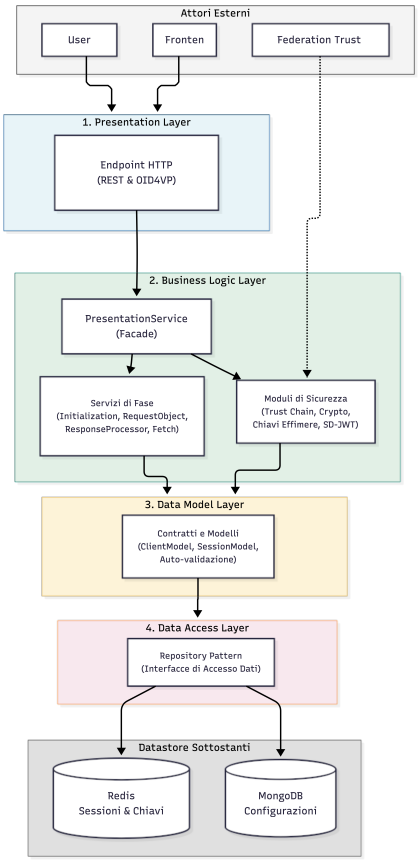


Figura 3.1: Architettura generale del sistema Relying Party

3.1.1 Stratificazione architetturale

Per mantenere il codice pulito e facile da gestire, e per separare bene le responsabilità, il sistema si basa su una struttura a strati orizzontali, chiamata *Layered Architecture*. Come mostra la Figura 3.1, l'architettura è divisa in quattro livelli principali, posti tra gli attori esterni (interfacce client e Digital Wallet) e i sistemi di persistenza.

3.1.1.1 Presentation Layer (Presentazione e instradamento)

Il *Presentation Layer* rappresenta, di fatto, l'unico punto attraverso cui un client esterno può raggiungere la logica interna del sistema. Non si tratta di un livello in cui risiedono regole di business: il suo compito è più simile a quello di un instradatore e adattatore di protocollo. In concreto, riceve le richieste HTTP_G e le smista a seconda della loro natura; da un lato le chiamate REST_G legate alla configurazione dei client, dall'altro gli endpoint che i Digital Wallet interrogano durante il ciclo di vita della presentazione OID4VP.

Una volta che la richiesta è stata accolta, il layer ne analizza il payload dal punto di vista sintattico e si fa carico della normalizzazione dei formati. Questo passaggio è necessario perché le specifiche $\text{OAuth 2.0}_\text{G}$ e OID4VP utilizzano nomenclature in *snake_case*, mentre il codice TypeScript del progetto adotta internamente la convenzione *camelCase*; il layer si occupa dunque di tradurre i nomi dei campi dall'una all'altra. Oltre a ciò, qui viene centralizzata anche la gestione delle eccezioni: gli errori che risalgono dai livelli inferiori vengono intercettati e trasformati in risposte HTTP ben formate, ciascuna associata al codice di stato pertinente (ad esempio `400`, `404`, `412` oppure `500`).

3.1.1.2 Business Logic Layer (Logica applicativa e servizi)

Il *Business Logic Layer* rappresenta il cuore computazionale dell'intera applicazione. È qui che trovano posto le regole previste dal protocollo OID4VP (di cui si è parlato nella Sezione 1.2) insieme a tutte le logiche dedicate alla convalida crittografica. Si è scelto di non concentrare questa logica in classi monolitiche: si è preferito invece adottare il pattern *Facade* abbinato a una pipeline di elaborazione organizzata per fasi. In concreto, la classe `PresentationService` svolge il ruolo di coordinatore centrale, offrendo al livello di instradamento un'interfaccia unica e delegando poi l'esecuzione del flusso transazionale ai singoli servizi di fase, ovvero `InitializationService`, `RequestObjectService`, `ResponseProcessorService` e `FetchService`.

Accanto a questi servizi di fase lavorano alcuni moduli trasversali con compiti specifici sul piano della sicurezza. Tra le loro responsabilità rientrano la risoluzione ricorsiva delle catene di fiducia (*Trust Chains*$_{\mathrm{G}}$), condotta in conformità alle specifiche OpenID Federation 1.0, la negoziazione delle suite crittografiche disponibili e la generazione di coppie di chiavi asimmetriche effimere, necessarie ad assicurare la *Forward Secrecy*$_{\mathrm{G}}$. A questi si aggiunge l'analisi crittografica delle firme e delle *selective disclosures* presenti all'interno delle credenziali SD-JWT.

3.1.1.3 Data Model Layer (Modelli di dominio e contratti dati)

Il *Data Model Layer* ha il compito di definire il vocabolario tipizzato e i contratti dati che l'intera applicazione condivide. Grazie al sistema di tipi offerto da TypeScript, questo livello fissa la struttura logica di ciascun dato e i vincoli a cui esso deve sottostare prima di poter essere elaborato oppure persistito. Vi trovano posto tanto i modelli destinati alla persistenza, come `ClientModel`, che cattura la configurazione di un Verifier, quanto i modelli transazionali di natura volatile, come ad esempio `SessionModel`, il cui scopo è tenere traccia dello stato effimero e della query DCQL$_{\mathrm{G}}$ associata alla sessione in corso.

Un aspetto caratterizzante di questi modelli di dominio è la presenza di meccanismi di autovalidazione incorporati direttamente al loro interno: già in fase di costruzione dell'istanza, i costruttori si occupano di accertare che i campi obbligatori siano effettivamente valorizzati, che i formati URI siano conformi a quanto atteso e che gli identificativi soddisfino i requisiti di sicurezza previsti.

3.1.1.4 Data Access Layer (Accesso ai dati e persistenza)

Il *Data Access Layer* si occupa di astrarre e tenere separati i dettagli implementativi che riguardano la memorizzazione fisica delle informazioni. Attraverso l'adozione del *Repository pattern*, la logica di business interagisce unicamente con interfacce di alto livello, senza dover conoscere né gestire la complessità legata ai driver di persistenza sottostanti.

Questo layer coordina l'accesso a due tecnologie di database differenti ma tra loro complementari. Da un lato si trova un database NoSQL di tipo documentale (MongoDB), utilizzato per conservare in modo strutturato e multi-tenant le configurazioni dei client Verifier, con una prospettiva di lungo periodo. Dall'altro è presente un archivio in-memory di tipo chiave-valore (Redis), pensato per gestire con elevate prestazioni e in maniera atomica le sessioni transazionali, i record di risposta temporanei e le chiavi crittografiche effimere, tutti elementi caratterizzati da una scadenza controllata tramite *Time-To-Live*.

3.2 Organizzazione della codebase



Figura 3.2: Organizzazione della codebase

3.2.1 Criteri di modularità e convenzioni

Quando si progetta un sistema che impone vincoli crittografici e di interoperabilità così stringenti, la struttura del codice non può essere lasciata al caso. In quest'ottica, il progetto si affida a una decomposizione di tipo ibrido:

- Al primo livello, la radice del sorgente (`src/`) ripartisce i moduli secondo il livello logico a cui appartengono, separando nettamente `routes`, `services`, `models` e `repositories`.

- Dentro ogni singolo layer, poi, i file vengono raggruppati ulteriormente in base alla funzionalità specifica o alla fase del protocollo di cui si occupano.

Il principio di singola responsabilità (*Single Responsibility Principle*) è rispettato in maniera tassativa: ciascun modulo TypeScript incapsula una sola classe, interfaccia o funzione principale e opera entro un dominio applicativo chiaramente circoscritto.

Sul piano delle convenzioni di naming, i file sorgente hanno suffissi semantici espliciti (ad esempio `*.route.ts`, `*.service.ts`, `*.model.ts`, `*.repository.ts`). In questo modo, il ruolo architetturale di un modulo si desume immediatamente dalla sua estensione, senza ambiguità.

3.2.2 Decomposizione logica dei moduli per layer

La distribuzione delle responsabilità dentro la directory `src/` si articola sui livelli seguenti:

3.2.2.1 Layer dei modelli di dominio (`src/models/`)

Qui i modelli non sono semplici contenitori passivi di dati: si tratta di vere e proprie entità di dominio, dotate di tipizzazione rigorosa e capaci di auto-validarsi.

- **`ClientModel`:** descrive le capacità operative e i vincoli crittografici di ogni Verifier registrato. Attraverso enumeratori tipizzati specifica gli algoritmi di firma JWS ammessi (ad esempio `ES256`, `RS256`), quelli di cifratura JWE (come `RSA-OAEP-256`), i set di chiavi JWKS e gli endpoint autorizzati.
- **`VpFormatsSupported`:** raccoglie in forma aggregata ed estensibile i formati di Credenziali Verificabili supportati, includendo specializzazioni come `SdJwtFormatSupported` (che parametrizza SD-JWT e Key Binding JWT).
- **`SessionModel`:** governa lo stato transazionale effimero presente su Redis. Oltre a conservare la query DCQL, il *nonce* crittografico e gli identificativi di transazione, questo modello effettua controlli di sicurezza preventivi già in fase di istanziazione: blocca la creazione di sessioni il cui identificativo risulti non conforme o privo di codifica *URL-safe*.

3.2.2.2 Layer di presentazione e instradamento (`src/routes/`)

Il modulo `presentation.route.ts` è volutamente privo di logica di business, così da preservare la separazione delle responsabilità. Il suo compito è gestire l'interfaccia HTTP del sistema:

- definisce e registra gli endpoint REST in conformità alle specifiche OID4VP;
- provvede alla sanitizzazione preliminare dei parametri in ingresso (body, query string o path param) per poi instradarli ai metodi del layer di servizio sottostante.

3.2.2.3 Layer di logica applicativa (`src/services/`)

Questo è il cuore computazionale del sistema, organizzato in sotto-moduli altamente specializzati:

- **Orchestratore principale (`presentation.service.ts`):** applica il pattern *Facade* e offre un punto di accesso unificato attraverso cui coordinare l'intero ciclo di vita della presentazione.
- **Fasi transazionali OID4VP:** sotto-directory dedicate (`initialization`, `request-object`, `response-processor`, `fetch`) isolano la logica propria di ciascuna fase del protocollo.
- **Validazione polimorfica (`prefix-validator.service.ts`):** seleziona dinamicamente la strategia di validazione del client in funzione del prefisso del Client ID

impiegato (`redirect_uri`, certificati X.509, `openid_federation`, `verifier_attestation`).

- **Sicurezza e crittografia:** i moduli nelle cartelle `decipher` e `verifier` racchiudono le operazioni crittografiche (decifratura JWE con chiavi effimere, verifica delle firme SD-JWT, convalida dell'*Holder Binding*), tenendo la complessità computazionale separata dalla gestione del flusso applicativo.

3.2.2.4 Layer di accesso ai dati (`src/repositories/`)

Questo layer incapsula ogni interazione con i sistemi di storage, seguendo il *Repository pattern*:

- **`mongodb-client.repository.ts`:** si occupa della persistenza a lungo termine delle configurazioni client su MongoDB. Eredita da interfacce conformi allo standard SCIM 2.0, così da supportare nativamente la gestione multi-tenant delle identità.
- **`redis-session.repository.ts`:** astrae la gestione della memoria volatile su Redis, nascondendo l'esecuzione di comandi atomici a basso livello come `GETDEL`, impiegato per la cancellazione sicura delle chiavi effimere.

3.3 Modellazione dei dati di dominio

Il *Data Model Layer* costituisce la base logica del sistema. In questo livello, i dati eterogenei provenienti dall'esterno vengono validati e mappati in entità TypeScript fortemente tipizzate.

3.3.1 Entità persistenti: il modello ClientModel e le politiche di sicurezza

A differenza di contesti applicativi centrati sull'utente finale, qui l'entità `ClientModel` rappresenta l'organizzazione che riveste il ruolo di verificatore (*Verifier*). Funziona, in pratica, come un dizionario centralizzato delle politiche di sicurezza, delle identità digitali e dei vincoli crittografici che il Relying Party applica alle sessioni di presentazione:

- **URI di reindirizzamento autorizzati:** elenca gli endpoint di redirect ammessi per il client (`redirectUris`), scongiurando vulnerabilità di tipo *Open Redirect* e tentativi di deviare il flusso di autorizzazione verso domini malevoli;
- **Capacità e algoritmi crittografici negoziabili:** proprietà dedicate dichiarano gli algoritmi supportati (suite di firma JWS_G, cifratura asimmetrica JWE_G e così via), delimitando a priori gli schemi crittografici consentiti per ciascun partner. Il sistema può così bloccare sul nascere l'interazione con wallet che propongano suite deboli, vulnerabili o non allineate ai profili di interoperabilità EUDI Wallet;
- **Set di chiavi crittografiche pubbliche (JWKS_G):** tramite la struttura `jwks` (*JSON Web Key Set*), l'entità conserva le chiavi pubbliche ufficiali del client, indispensabili per l'esposizione nei metadati e per la firma digitale del *Request Object*.

3.3.2 Strutture transazionali ed effimere: query DCQL e VP Token_G

Il protocollo OID4VP poggia sulla possibilità, per il Verifier, di esprimere con granularità i requisiti di presentazione delle credenziali. Questa esigenza trova formalizzazione nel modello `DCQLQuery` (*Digital Credentials Query Language*), una struttura dati integrata nello stato di sessione che stabilisce i criteri di ammissibilità della richiesta:

- **Tipologia di credenziale (`vct`):** identifica univocamente il tipo di credenziale richiesto (si pensi ad attestati di identità digitale o patenti di guida);

- **Percorsi dei claim (*claim paths*):** indica i singoli attributi soggetti a rivelazione selettiva (*selective disclosure*), evitando di condividere in blocco dati non pertinenti e rispettando così il principio di minimizzazione;
- **Autorità emittenti fidate (*trusted issuers*):** vincola l'accettazione delle presentazioni a emettitori autorizzati e riconosciuti;
- **Insiemi di credenziali alternativi (`credentialSets`):** permette di definire condizioni alternative, consentendo all'utente di soddisfare la richiesta con diverse combinazioni di credenziali.

Nel momento in cui il Wallet trasmette la risposta all'endpoint designato, le credenziali fornite vengono incapsulate nell'oggetto `VpToken`. Questa entità non si limita a memorizzare le stringhe crittografiche ricevute: costituisce piuttosto una mappa relazionale che correla ogni presentazione (*Verifiable Presentation*) con il corrispondente identificativo di query definito nella `DCQLQuery` originaria. Grazie a tale correlazione, il motore di verifica è in grado di validare in modo deterministico e ordinato la corrispondenza della presentazione ai requisiti fissati in partenza.

3.3.3 Contesti di validazione e Data Transfer Object (DTO)

A completare il modello dei dati, il layer include strutture intermedie e oggetti di trasferimento (*Data Transfer Object*, $\mathrm{DTO_G}$) pensati per veicolare le informazioni lungo la pipeline dei servizi applicativi in modo sicuro e normalizzato.

Un elemento particolarmente significativo è il modello `PrefixValidationResult`. Nel protocollo OID4VP, infatti, un client o un wallet può accreditarsi attraverso schemi di identificazione molto diversi tra loro: URI pre-registrati, catene di certificati digitali X.509, attestazioni crittografiche di tipo *Verifier Attestation*, oppure federazioni OpenID. Il servizio di validazione preliminare del prefisso prende in carico questo input eterogeneo e restituisce un'istanza normalizzata di `PrefixValidationResult`. Grazie a questa astrazione, i layer a valle possono operare senza curarsi della modalità di identificazione originaria, fondando le successive decisioni operative esclusivamente sui vincoli formali esposti dal contesto (ad esempio l'obbligo o meno della firma digitale sul *Request Object*, oppure la determinazione dell'algoritmo negoziato).

3.4 Strategia di persistenza e Data Access Layer

Il sistema utilizza un'architettura di persistenza basata su tecnologie diverse, scelte in base al ciclo di vita e alle prestazioni richieste dai dati. Questo approccio separa i dati transazionali effimeri ad accesso frequente dallo storage persistente, dedicato alle configurazioni a lungo termine.

3.4.1 Storage volatile ad alte prestazioni: Redis

La gestione dello stato transazionale in tempo reale è affidata a Redis, impiegato come archivio dati in-memory ad alto rendimento. All'interno di tale datastore risiedono le sessioni OID4VP attive, le coppie di chiavi crittografiche effimere e i record di risposta temporanei (`ResponseRecord`).

L'interazione con Redis è mediata dall'astrazione del repository `RedisSessionRepository`, la quale applica sistematicamente il meccanismo di scadenza nativa basato su *Time-To-Live* (TTL). Ogni istanza di sessione o record transazionale è vincolata a un tempo di vita prefissato (e.g., dieci minuti): qualora un utente interrompa anticipatamente il flusso di presentazione sul Wallet, i relativi dati scadono e vengono rimossi automaticamente dal motore di Redis. Tale comportamento elimina l'accumulo di record orfani senza richiede-

re processi periodici di bonifica (*garbage collection*) in background, riducendo l'overhead computazionale e prevenendo la saturazione della memoria.

Per prevenire condizioni di competizione (*race conditions*) e tentativi di riutilizzo della sessione (*replay attack*$_{\text{G}}$), il ciclo di vita della presentazione è regolato da transizioni di stato atomiche: il metodo `consumeSessionForResponse()` verifica che lo stato della sessione sia `pending` e ne applica contestualmente la mutazione a `response_received`, sfruttando le primitive monothread di Redis. Qualsiasi richiesta concorrente sul medesimo identificativo viene immediatamente rigettata.

3.4.2 Storage persistente e multi-tenant: MongoDB

La persistenza a lungo termine delle entità di configurazione permanente (quali i modelli `ClientModel` e i relativi metadati) è delegata a MongoDB, database NoSQL orientato ai documenti. Tale tecnologia risulta particolarmente idonea alla rappresentazione di strutture gerarchiche e annidate complesse in formato BSON/JSON, in piena aderenza agli schemi previsti dallo standard SCIM 2.0.

L'accesso ai dati è incapsulato dal modulo `MongoDBClientRepository`, il quale implementa il *Repository pattern* al fine di disaccoppiare la logica applicativa dai driver nativi del database. Il repository sfrutta indici composti univoci (in particolare sulla coppia identificativa `domainId` e `clientId`) per assicurare l'integrità referenziale e prevenire collisioni tra registrazioni concorrenti. Tale impostazione garantisce il rigido isolamento logico dei dati tra tenant distinti direttamente a livello di motore di persistenza.

3.5 Sicurezza crittografica e gestione della fiducia

L'infrastruttura del Relying Party integra un'architettura di sicurezza orientata alla difesa finalizzata a mitigare attacchi di furto d'identità, manomissione dei dati, intercettazione delle comunicazioni e riproduzione fraudolenta (*replay attack*).

3.5.1 Forward secrecy applicativa mediante JWE e chiavi effimere

Al fine di garantire la proprietà di *Forward Secrecy*, il sistema non fa ricorso a chiavi crittografiche asimmetriche statiche memorizzate permanentemente sul server. All'atto della generazione della richiesta per il Wallet, il modulo genera dinamicamente una coppia di chiavi effimere: la componente pubblica viene inclusa all'interno del *Request Object* inviato al Wallet, che la impiega per cifrare il payload di risposta contenente le credenziali dell'utente; la componente privata corrispondente viene memorizzata temporaneamente su Redis con un TTL limitato.

All'atto della ricezione della risposta cifrata, il servizio `DecipherJWEService` preleva la chiave privata ed esegue l'operazione atomica `GETDEL`, eliminandola istantaneamente dal database. La distruzione irreversibile della chiave garantisce che, anche nell'eventualità di una successiva compromissione delle chiavi di lungo periodo del Verifier o dell'intercettazione prolungata del traffico di rete, la decifratura a posteriori dei dati risulti matematicamente inattuabile.

3.5.2 Risoluzione gerarchica della fiducia: OpenID Federation 1.0 e Trust Chains

La validazione di credenziali e attori si basa sul modello *Trust Chain*$_{\text{G}}$ di OpenID Federation 1.0. In questo schema la fiducia non è mai presunta, ma viene costruita e verificata tramite una catena gerarchica di attestazioni digitali.

Qualora sia richiesta la validazione di un'entità, il componente `TrustAnchorService` risale la gerarchia federativa attraverso un algoritmo di visita in profondità sui grafi, integrando

controlli specifici per inibire cicli infiniti o condizioni di stallo. Il servizio recupera l'*Entity Configuration* pubblica dell'entità target, interroga le autorità intermedie indicate nei relativi *authority hints* per ottenerne le dichiarazioni subordinate (*Subordinate Statements*) e verifica ricorsivamente le firme digitali di ciascun livello, arrestandosi al raggiungimento di una radice di fiducia (*Trust Anchor*) pre-configurata e validata *out-of-band*.

3.5.3 Validazione crittografica di SD-JWT e prevenzione dei Replay Attack

All'atto della ricezione del pacchetto di risposta (VP Token in formato SD-JWT), i moduli `PresentationValidatorService` e `SdJwtCredentialVerifierService` eseguono una pipeline di verifiche crittografiche sequenziali:

- **Verifica dell'emittente (*Issuer Verification*):** convalida della firma digitale sull'Issuer JWT, recuperando le chiavi pubbliche dell'emittente tramite la relativa *Trust Chain*;
- **Verifica dell'integrità delle rivelazioni (*Hash Checking*):** nell'architettura SD-JWT l'utente può esercitare la divulgazione selettiva (*selective disclosure*) dei propri attributi. Il sistema calcola l'impronta crittografica SHA-256 di ciascun claim divulgato e ne accerta la perfetta corrispondenza con i digest originariamente inseriti dall'emittente nel documento inalterabile;
- **Convalida dell'Holder Binding e Proof of Possession:** oltre alla validità intrinseca della credenziale, il sistema verifica che il soggetto presentatore sia l'effettivo titolare. Tale controllo avviene validando la firma di un secondo token JWT (*Key Binding JWT*, KB-JWT_{G}) generato dal dispositivo dell'utente con la chiave crittografica associata alla credenziale medesima;
- **Mitigazione dei Replay Attack_{G}:** la convalida del KB-JWT include la verifica rigorosa della presenza del *nonce* monouso emesso dal Verifier e della corretta valorizzazione dell'*audience*, inibendo qualsiasi tentativo di riutilizzo fraudolento di sessioni di presentazione precedentemente intercettate.

L'esecuzione coordinata di tali controlli assicura che i dati recapitati all'interfaccia frontend del Relying Party posseggano valore legittimo, attestando in modo inequivocabile l'identità digitale e il consenso informato dell'utente.

Capitolo 4

Codifica e implementazione

In questo capitolo viene descritta la fase di codifica e implementazione del sistema, condotta sulla base della progettazione illustrata nel Capitolo 3. Vengono presentati la metodologia di sviluppo adottata, i flussi operativi concreti del protocollo OID4VP$_{\text{G}}$ con i relativi frammenti di codice e le sfide tecniche affrontate durante l'implementazione.

4.1 Metodologia e organizzazione del lavoro

4.1.1 Studio del contesto e degli standard

La fase di codifica è stata preceduta da un'attività di studio approfondito del contesto normativo e tecnico in cui il sistema opera. Trattandosi di un componente crittografico inserito in un ecosistema di identità decentralizzata, ogni scelta implementativa è vincolata a specifiche sezioni delle relative RFC e specifiche tecniche. I documenti di riferimento che hanno guidato l'intera fase sono stati:

- la specifica **OpenID for Verifiable Presentations** (OID4VP), che definisce il protocollo di interazione tra Verifier e Wallet;
- lo standard **SD-JWT$_{\text{G}}$** (*Selective Disclosure JWT*), che introduce un meccanismo basato su *hash* crittografici per la rivelazione selettiva degli attributi;
- il linguaggio **DCQL$_{\text{G}}$** (*Digital Credentials Query Language*), che formalizza la modalità con cui un Verifier esprime richieste complesse di credenziali;
- il framework **OpenID Federation 1.0**, che definisce la costruzione e la validazione delle catene di fiducia tra entità mediante la navigazione ricorsiva di *Entity Configuration* e *Subordinate Statement*;
- le specifiche **JAR$_{\text{G}}$** (*JWT-Secured Authorization Request*), che prescrivono l'incapsulamento della richiesta di autorizzazione in un JWT$_{\text{G}}$ firmato digitalmente.

Parallelamente, è stata condotta un'analisi del framework interno Monokee (le librerie `@monokee-dev` descritte nella Sezione 1.2) per comprendere le astrazioni preesistenti su cui costruire il sistema. Il framework fornisce classi base per la creazione di server Express con supporto nativo al protocollo SCIM$_{\text{G}}$ 2.0 e un pattern di repository generico per MongoDB.

4.1.2 Approccio incrementale *bottom-up*

L'approccio di sviluppo adottato è stato di tipo incrementale e *bottom-up*, partendo dalle fondamenta del sistema e risalendo progressivamente verso le interfacce esposte. Le fasi di sviluppo hanno seguito la stratificazione architetturale descritta nella Sezione 3.2:

1. **Modellazione dei dati:** definizione delle entità di dominio (`ClientModel`, `SessionModel`, `DCQLQuery`, `ResponseRecord`) e degli enumeratori crittografici che vincolano

a livello di tipo i valori ammissibili per gli algoritmi. Questa scelta ha permesso di stabilire un contratto formale condiviso prima di implementare la logica applicativa.

2. **Layer di accesso ai dati:** implementazione del `RedisSessionRepository` e del `MongoDBClientRepository`, consentendo la verifica immediata della connettività e della serializzazione delle entità sui datastore reali.
3. **Layer di logica applicativa:** implementazione dei servizi di fase (inizializzazione, *Request Object*, elaborazione della risposta, *fetch*), dei servizi di validazione crittografica e dei servizi di cifratura. Ogni servizio è stato sviluppato e verificato in isolamento prima dell'integrazione nel flusso complessivo.
4. **Layer di presentazione:** implementazione delle route Express, del middleware di gestione errori e dell'integrazione finale nell'*entry point* `app.ts`.

4.2 Flussi operativi del protocollo OID4VP

Le sezioni seguenti descrivono l'implementazione concreta dei flussi operativi del protocollo, facendo riferimento alla progettazione architetturale e ai pattern descritti nel Capitolo 3.

4.2.1 Inizializzazione della sessione

Il flusso operativo inizia quando un *frontend* invoca l'endpoint `POST /presentation/init`. L'`InitializationService`, operando all'interno della pipeline a fasi descritta nella Sezione 3.2, gestisce questa fase con la seguente sequenza di operazioni:

1. recupero da MongoDB della configurazione del client (`ClientModel`) associato al `domainId` e al `clientId` forniti; in caso di assenza, l'operazione viene bloccata con un `ForbiddenError`;
2. analisi del prefisso del `clientId` per determinare lo schema di identificazione da adottare; qualora il prefisso indichi `openid_federation`, viene attivato un sotto-flusso critico di risoluzione e validazione della Trust Chain$_G$, invocando il `TrustAnchorService` per percorrere ricorsivamente la catena di fiducia fino a un *Trust Anchor* noto;
3. creazione di una nuova sessione tramite il `SessionService`, con generazione di quattro identificativi crittograficamente casuali (`sessionId`, `requestId`, `transactionId`, `nonce`) e salvataggio in Redis con un TTL$_G$ configurabile;
4. costruzione di un URL di autorizzazione (`authUrl`): se il prefisso non richiede firma (ad esempio `pre_registered`), l'URL contiene tutti i parametri come *query string*; se la richiede (ad esempio `x509_san_dns`), l'URL contiene un riferimento a un `request_uri` da cui il Wallet scaricherà il JWT firmato.

```
curl -X POST https://dronish-unlabiate-jaylah.ngrok-free.dev/d9f7b5ac-ce99-4757-94b5-0de0a60ad471/Clients/c03d9acc-cd36-473b-a3bf-f7d7eed826d6/presentation/init \
-H "Content-Type: application/json" \
-d '{
"prefixClientId": "x509_san_dns",
"responseMode": "direct_post.jwt",
"dcqlQuery": {
  "credentials": [
    {
      "id": "eudi_pid_identity",
      "format": "dc+sd-jwt",
      "meta": {
        "vct_values": [
          "urn:eu.europa.ec.eudi:pid:1",
          "urn:eudi:pid:1"
        ]
      },
      "claims": [
        { "path": ["family_name"] },
        { "path": ["given_name"] }
      ]
    }
  ]
}
}'
```

Figura 4.1: Flusso di inizializzazione della sessione di presentazione

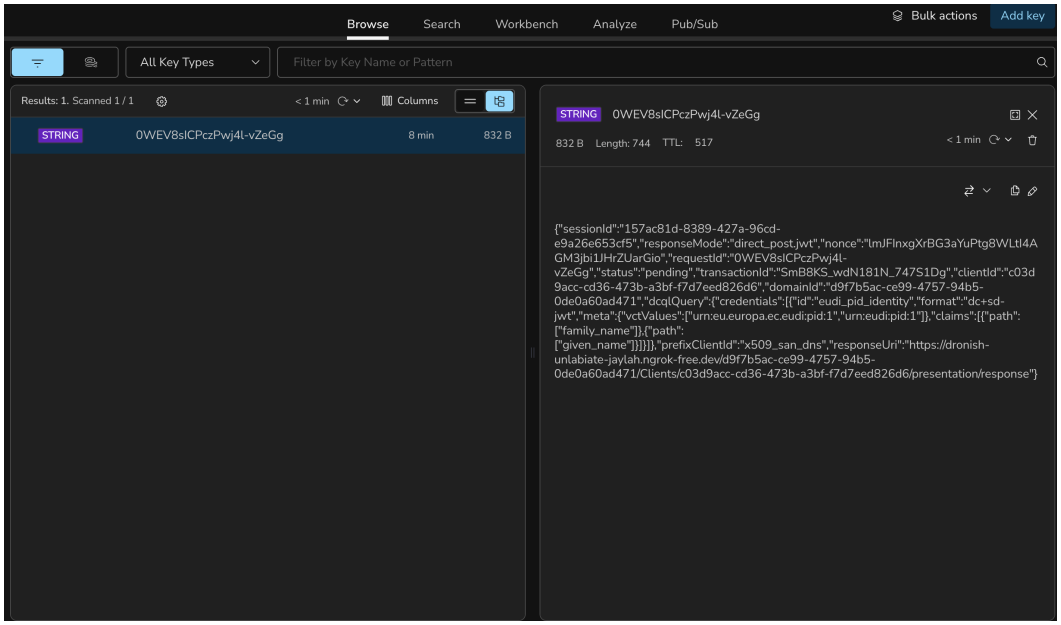


Figura 4.2: Sessione salvata su Redis

4.2.2 Generazione del Request Object

Quando il Wallet scansiona il QR Code o segue il *deep link*, contatta il backend all'endpoint `GET/POST /request/:requestId`. Il `RequestObjectService` orchestra una pipeline articolata in cinque step:

1. **Controllo anti-replay:** verifica che il flag `isRequestUriConsumed` sulla sessione sia ancora `false`. Se il *Request Object* è già stato scaricato in precedenza, la richiesta viene rifiutata con un `ForbiddenError`.
2. **Negoziazione algoritmica:** il `VerifierService` confronta le capacità crittografiche dichiarate dal Wallet nel proprio `wallet_metadata` con quelle configurate nel `ClientModel`. Viene ricercata un'intersezione tra gli algoritmi JWE_G supportati, gli algoritmi di firma JWS_G e i formati VP. In assenza di compatibilità, il servizio segnala l'errore con codici semantici (`alg_not_supported`, `enc_not_supported`, `vp_formats_not_supported`).
3. **Generazione delle chiavi effimere:** attivato solo se la negoziazione JWE ha avuto esito positivo. L'`EphemeralKeyService` genera una coppia di chiavi asimmetriche (EC P-256 o RSA 2048-bit, a seconda dell'algoritmo negoziato), ne salva la parte privata in Redis con un TTL e inserisce la chiave pubblica JWK_G nel `client_metadata` del *Request Object*. Il Wallet utilizzerà questa chiave pubblica per cifrare la propria risposta, secondo il meccanismo di *Forward Secrecy*$_\text{G}$ descritto nella Sezione 3.5.
4. **Costruzione dell'Entity Configuration** (specifico per il prefisso `openid_federation`): il servizio assembla un JWT auto-firmato che descrive il Verifier nell'ecosistema federato, includendo le proprie chiavi pubbliche, gli `authority_hints` e i metadati del Relying Party.
5. **Firma digitale del JWT:** il servizio importa la chiave privata dal JWKS del client, verifica la corrispondenza tra chiave privata e certificato X.509 (se presente nell'header `x5c`), e firma il JWT *Request Object* con l'algoritmo negoziato. Per i client `pre_registered`, viene generato un *Unsecured JWT* (con `alg:"none"` e firma vuota), costruito manualmente in *base64url*.



Figura 4.3: Generazione del Request Object

4.2.3 Elaborazione della risposta del Wallet

Quando il Wallet invia la *Verifiable Presentation* all'endpoint `POST /presentation/response`, il `ResponseProcessorService` deve gestire due formati di input: la risposta può arrivare in chiaro (campi `vp_token` e `state` separati) oppure in formato cifrato (un campo `response` contenente un JWE *compact serialization* a cinque segmenti).

Nel caso cifrato, il servizio attiva il `DecipherJWEService`, che estrae il `kid` dall'header protetto del JWE, invoca `EphemeralKeyService.retrieveAndDestroyPrivateKey()` per ottenere atomicamente la chiave privata da Redis (tramite il comando `GETDEL`, descritto nella Sezione 3.4), e decifra il payload con `jose.compactDecrypt()`.

Una volta ottenuto il payload in chiaro, il servizio consuma atomicamente la sessione portandola dallo stato `pending` a `response_received` e avvia la fase di validazione crittografica, delegandola al `PresentationValidatorService` secondo la pipeline descritta nella Sezione 3.5.

Se la validazione ha esito positivo, viene creato un `ResponseRecord` contenente il VP Token_G validato e un `responseCode` monouso generato con 32 byte casuali. Il Wallet riceve un URI di *redirect* con questo codice nell'*hash fragment* (`#response_code=...`).

4.2.4 Ritiro dei dati verificati

Il `FetchService` implementa un protocollo di consumo monouso. Quando il *frontend* invoca `POST /presentation/fetch`, il servizio recupera la sessione e il `ResponseRecord` associato, verifica che il `responseCode` fornito corrisponda a quello salvato, controlla che il record non sia già stato consumato (`state !== 'consumed'`), e consegna il VP Token al chiamante. Infine, elimina la sessione da Redis, garantendo che ogni presentazione possa essere ritirata una e una sola volta.

4.2.5 Validazione crittografica SD-JWT e DCQL

Il `SdJwtCredentialVerifierService` implementa una pipeline di verifica per il formato SD-JWT. Il processo inizia con il *parsing* della presentazione: la stringa SD-JWT viene suddivisa lungo il separatore `~`, producendo un `issuerJwt`, un array di `disclosures` e un opzionale `keyBindingJwt` (KB-JWT_G).

La verifica della **firma dell'emittente** risolve la chiave pubblica dell'Issuer tramite il `TrustAnchorService` (se le autorità fidate sono di tipo `openid_federation`) o tramite l'header `jwk`/`x5c` del JWT, e valida crittograficamente la firma.

La verifica dei **hash delle *disclosure*** è l'operazione più caratteristica del formato SD-JWT: il servizio raccoglie ricorsivamente tutti i digest presenti negli array `_sd` del payload JWT dell'Issuer, calcola l'hash *base64url* di ciascuna *disclosure* presentata dall'utente e verifica che il risultato sia contenuto nell'insieme dei digest autorizzati. Se una *disclosure* non corrisponde ad alcun digest, l'intera presentazione viene rifiutata.

La verifica del **KB-JWT** attesta la titolarità della credenziale: il servizio importa la chiave pubblica dell'*Holder* dal claim `cnf.jwk` del payload dell'Issuer e verifica la firma del KB-JWT, controllando che l'*audience* corrisponda all'identificativo del Verifier e che il *nonce* corrisponda a quello generato nella fase di inizializzazione.

Il `PresentationValidatorService` opera a livello superiore e implementa la specifica DCQL. La validazione include:

- il controllo del `vct` (*Verifiable Credential Type*) contro i valori attesi;
- la valutazione di insiemi alternativi di credenziali (`credentialSets`), dove ogni set interno rappresenta un AND logico e i set esterni sono in OR tra loro;
- la gestione di `claimSets` alternativi, dove se una combinazione di claim non è presente viene tentata la successiva.

4.2.6 Risoluzione delle Trust Chain

Il `TrustAnchorService` implementa un algoritmo di *Depth-First Search* ($\mathrm{DFS_G}$) con rilevamento di cicli per costruire le catene di fiducia nell'ecosistema OpenID Federation.

L'algoritmo parte dall'*Entity ID* dell'Issuer e recupera la sua *Entity Configuration* dall'endpoint standard `/.well-known/openid-federation`. Da questa configurazione estrae gli `authority_hints` (i garanti dichiarati). Per ciascun garante, il servizio contatta il relativo `federation_fetch_endpoint` richiedendo il *Subordinate Statement* che lega il garante all'entità corrente. Se il garante non è un *Trust Anchor* noto, il processo si ripete ricorsivamente, risalendo la gerarchia fino a raggiungere un'entità le cui chiavi pubbliche ($\mathrm{JWKS_G}$) siano già pre-caricate in Redis come fiducia *out-of-band*.

Per prevenire cicli infiniti in grafi potenzialmente malformati, l'algoritmo mantiene un `Set<string>` di entità già visitate.

Una volta costruita la catena, il servizio esegue una validazione crittografica bidirezionale: dalla cima (il *Trust Anchor*, la cui firma viene verificata confrontando i JWKS salvati in Redis) verso il basso (ogni *Subordinate Statement* viene verificato con le chiavi dell'entità superiore). Infine, viene verificata l'auto-firma dell'*Entity Configuration* del nodo foglia e la corrispondenza del `sub` tra la foglia e il primo *Subordinate Statement*.

4.3 Sfide tecniche e soluzioni adottate

4.3.1 Atomicità del consumo di sessione in un sistema distribuito

Problema. In un sistema con molteplici istanze backend, la medesima sessione potrebbe essere consumata contemporaneamente da richieste concorrenti (*race condition*), causando la duplicazione del VP Token consegnato al *frontend*.

Soluzione. Il metodo `consumeSessionForResponse()` del `RedisSessionRepository` implementa un controllo atomico: legge la sessione, verifica che lo stato sia ancora `pending` e lo aggiorna in un'unica operazione. Qualsiasi tentativo concorrente troverà lo stato già mutato e verrà respinto con un `NotFoundError`.

4.3.2 Forward Secrecy e distruzione sicura delle chiavi private

Problema. La chiave privata effimera generata per la cifratura JWE deve essere utilizzata una sola volta e successivamente distrutta, per garantire la proprietà di *Forward Secrecy* (Sezione 3.5). È necessario impedire un secondo utilizzo in scenari concorrenti.

Soluzione. L'`EphemeralKeyService` sfrutta il comando Redis `GETDEL`, che legge e cancella il valore in un'unica operazione atomica:

```
const keyData = await redisClient.getdel(`ephemeral_key:${kid}`)
if (!keyData) {
  throw new NotFoundError('Ephemeral key not found or already consumed')
}
```

Listing 1: Utilizzo del comando Redis GETDEL per la rimozione atomica delle chiavi effimere

Se un secondo tentativo di decifratura interviene durante l'elaborazione del primo, il `GETDEL` troverà il dato già cancellato e solleverà un'eccezione. Ogni chiave è inoltre protetta da un TTL Redis che ne garantisce l'auto-distruzione anche in assenza di un *consumer*.

4.3.3 Verifica della corrispondenza chiave privata/certificato X.509

Problema. Per gli schemi di identificazione `x509_san_dns` e `x509_hash`, il Verifier deve firmare il *Request Object* con la chiave privata associata al certificato X.509 contenuto nel JWKS. Non esisteva alcuna garanzia automatica di corrispondenza tra la chiave privata e il certificato foglia nella catena `x5c`.

Soluzione. Nel `RequestObjectService`, prima della firma, il servizio esegue una verifica esplicita della corrispondenza tra le coordinate della chiave privata e quelle del certificato. Per le chiavi EC, vengono confrontate le coordinate `x` e `y`; per le chiavi RSA, il modulo `n` e l'esponente `e`:

```
if (privateKeyJwk.kty === 'EC' &&
    (privateKeyJwk.x !== certPublicKeyJwk.x || privateKeyJwk.y !==
    ↪  certPublicKeyJwk.y)) {
  throw new InvalidValueError(
    'Key mismatch: The private key does not correspond to the leaf
    ↪  certificate '
    + 'in the x5c chain.'
  )
}
```

Listing 2: Verifica della corrispondenza tra chiave privata e certificato X.509

Questa verifica preventiva impedisce che un Wallet riceva un JWT firmato con una chiave incoerente, evitando errori difficilmente diagnosticabili in ambiente di produzione.

4.3.4 Workaround per hostname Docker in ambiente di sviluppo

Problema. Durante lo sviluppo locale con Docker, il Wallet e il backend comunicano tramite l'hostname `host.docker.internal`, che non compare nel *Subject Alternative Name* (SAN) dei certificati X.509 di test (tipicamente limitati a `localhost`). Questo causava il fallimento sistematico della validazione `x509_san_dns` in ambiente di sviluppo.

Soluzione. Nel `ClientIdPrefixValidator` è stato inserito un *workaround* condizionale, attivo esclusivamente in ambiente di sviluppo:

```
if (process.env.NODE_ENV === 'development' &&
    !matchingDnsName && responseUriFqdn === 'host.docker.internal') {
  matchingDnsName = dnsNames.find((dns) => dns === 'localhost')
}
```

Listing 3: Workaround per hostname Docker in ambiente di sviluppo

La guardia `process.env.NODE_ENV === 'development'` assicura che questo rilassamento della validazione non si attivi in ambienti di produzione o *staging*.

4.3.5 Conversione bidirezionale *camelCase*–*snake_case* con eccezioni

Problema. Il codice TypeScript interno adotta la convenzione *camelCase* (ad esempio `vpFormatsSupported`), mentre il protocollo OID4VP/OAuth$_{\text{G}}$ richiede *snake_case* (ad esempio `vp_formats_supported`). Tuttavia, la conversione non poteva essere una semplice trasformazione meccanica: alcune chiavi come `jwks` non devono essere convertite, e la struttura `vpFormatsSupported` contiene chiavi con trattini (`dc+sd-jwt`) che non seguono alcuna delle due convenzioni.

Soluzione. Il modulo `case-converter.util.ts` implementa un convertitore ricorsivo con una lista di eccezioni esplicite: le chiavi `jwks` vengono preservate intatte, e il mapping di `vpFormatsSupported` verso `dc+sd-jwt` viene gestito con logica dedicata anziché affidato alla conversione automatica.

4.3.6 Discriminazione tra payload in chiaro e payload cifrato

Problema. L'endpoint `POST /presentation/response` deve gestire trasparentemente sia risposte in chiaro (con campi `vp_token` e `state` separati) sia risposte cifrate (un unico campo `response` contenente un JWE *compact serialization*). Il JWE *compact*, una volta decifrato, può contenere a sua volta un JWT firmato o un semplice JSON.

Soluzione. Il `ResponseProcessorService` implementa una discriminazione euristica: se il campo `response` è una stringa con esattamente cinque segmenti separati da `.`, viene trattata come JWE *compact* e decifrata. Il payload risultante viene poi interpretato prima come `JSON.parse()` e, in caso di fallimento, come `jose.decodeJwt()`:

```
if (typeof jwtStr === 'string' && jwtStr.split('.').length === 5) {
  jwtStr = await this.decipherJWEService.decipherJWE(jwtStr)
}
let payload: any
try {
  payload = JSON.parse(jwtStr)
} catch (e) {
  payload = jose.decodeJwt(jwtStr)
}
```

Listing 4: Discriminazione tra payload in chiaro e payload cifrato nella risposta del Wallet

Capitolo 5

Verifica e validazione

L'ultima fase del periodo di sviluppo ha riguardato il processo di verifica e validazione del software prodotto.

La verifica è un processo orientato verso l'interno del prodotto, ovvero si occupa di fornire evidenza oggettiva che i risultati ottenuti come output di ciascuna fase dello sviluppo soddisfino i requisiti specificati per quella fase. Essa comprende attività come la revisione del codice, l'analisi statica, il *type checking*, i test unitari e i test d'integrazione.

La validazione, invece, è un processo orientato verso l'esterno, verso l'utente finale e il contesto d'uso: ha lo scopo di confermare in modo definitivo che le caratteristiche del software siano conformi ai bisogni dell'utente e all'uso previsto. La validazione richiede tipicamente l'esecuzione del sistema in condizioni che simulino l'uso reale. Nel caso di un software che implementa uno standard pubblico, la forma più autorevole di validazione è il superamento dei test di conformità (*conformance test*) rilasciati dall'ente che governa lo standard.

Nel caso del progetto `monokee-relyingparty-core`, la strategia di verifica e validazione ha combinato meccanismi di verifica statica, testing manuale dei flussi operativi e, come strumento di validazione formale, l'esecuzione dei Conformance Test della OpenID Foundation per il profilo OID4VP$_{\mathrm{G}}$ Verifier.

5.1 Verifica statica

5.1.1 Type checking e compilazione TypeScript

Il primo livello di verifica è fornito dal compilatore TypeScript stesso. L'intero progetto è scritto in TypeScript con configurazione `alwaysStrict:true`, il che impone al compilatore di verificare staticamente la coerenza dei tipi in ogni assegnazione, invocazione di metodo e accesso a proprietà. Qualsiasi errore di tipo viene intercettato a tempo di compilazione, prima ancora che il codice venga eseguito.

L'ampio impiego di enumeratori tipizzati (come `JWSAlgorithm`, `JWEAlgorithm`, `JWEEncAlgorithm`, `CdSdJwtAlg` definiti in `client.model.ts`) vincola i valori ammissibili per gli algoritmi crittografici a un insieme finito e chiuso. Ciò significa che un errore di battitura in un nome di algoritmo (ad esempio `"EC256"` invece di `"ES256"`) non può sfuggire alla fase di compilazione, eliminando un'intera classe di errori crittografici potenzialmente critici che coinvolgerebbero le suite JWS$_{\mathrm{G}}$ e JWE$_{\mathrm{G}}$.

Il comando `yarn typecheck` (definito in `package.json`) consente di eseguire la verifica dei tipi in modo isolato, senza produrre artefatti di build, facilitandone l'integrazione in pipeline di verifica continua.

5.1.2 Analisi statica con ESLint

Un secondo livello di verifica statica è garantito dalla configurazione di ESLint con il plugin `@typescript-eslint`, che analizza il codice alla ricerca di pattern problematici, variabili non utilizzate, importazioni circolari e violazioni delle best practice JavaScript/TypeScript. La configurazione Prettier (`.prettierrc`) garantisce inoltre la coerenza stilistica del codice, riducendo il rumore durante le revisioni e facilitando la leggibilità.

Il processo di sviluppo, come configurato nel file `nodemon.json`, esegue automaticamente `yarn lint` ad ogni riavvio del server di sviluppo, integrando così la verifica statica nel ciclo di feedback dello sviluppatore.

5.1.3 Audit automatico delle dipendenze

Il file `nodemon.json` integra un'ulteriore misura di verifica nel ciclo di sviluppo: il comando `yarn audit --groups dependencies --level high` viene eseguito automaticamente ad ogni riavvio del server. Questo comando interroga i database pubblici di vulnerabilità note per verificare che nessuna delle dipendenze del progetto contenga falle di sicurezza ad alta criticità. In un sistema che manipola dati di identità digitale e credenziali crittografiche, questa verifica è particolarmente rilevante.

5.1.4 Misure di protezione integrate nel codice

Accanto agli strumenti di verifica statica in senso stretto, il codice incorpora una serie di meccanismi difensivi che contribuiscono alla correttezza complessiva del sistema.

I costruttori dei modelli di dominio (`Session` e `Client`), come approfondito nella Sezione 3.2 del Capitolo 3, eseguono una validazione immediata dei dati in ingresso, verificando la presenza dei campi obbligatori e la conformità degli identificativi crittografici ai requisiti di sicurezza. In particolare, il metodo statico `isUrlSafe()`:

```
static isUrlSafe(text: string): boolean {
  return /^[a-zA-Z0-9._~-]+$/.test(text)
}
```

Listing 5: Metodo `isUrlSafe` per la validazione di stringhe URL-safe

impedisce l'inserimento di caratteri speciali nei parametri destinati a comparire in URL OAuth$_G$ e nei parametri di redirect, neutralizzando potenziali vulnerabilità di manipolazione degli indirizzi.

Per le operazioni CRUD$_G$ sui client, il framework SCIM$_G$ 2.0 applica automaticamente la validazione dei payload in ingresso confrontando gli schemi definiti in `client.schema.json`, garantendo che nessuna configurazione sintatticalmente invalida possa essere persistita in MongoDB.

Infine, la gerarchia di errori tipizzata (`InvalidValueError`, `ForbiddenError`, `NotFoundError`, `PreconditionFailedError`, `NotImplementedError`) e la relativa mappatura deterministica in codici HTTP$_G$, già descritta nel Capitolo 4, garantisce che ogni condizione anomala sia codificata esplicitamente e che gli errori interni restituiscano un messaggio generico, prevenendo la fuga accidentale di informazioni sull'architettura interna del sistema.

5.2 Verifica dinamica

5.2.1 Testing manuale dei flussi OID4VP

In assenza di una suite di test automatizzati, la verifica dinamica del comportamento del sistema è stata condotta tramite testing manuale end-to-end dei flussi OID4VP, utilizzando

client HTTP (Postman e cURL) e l'integrazione con l'ambiente di sviluppo locale (Docker, Redis, MongoDB e un tunnel ngrok per l'esposizione pubblica del servizio).

Per ciascuna delle quattro fasi del protocollo descritte nel Capitolo 4 sono stati verificati i flussi nominali e i principali scenari di errore:

- **Fase 1 (Init)**: corretta creazione della sessione, generazione degli identificativi crittografici, costruzione dell'`authUrl`, e rifiuto in caso di client inesistente o dominio non configurato;
- **Fase 2 (Request Object)**: corretta firma del JWT_{G}, inclusione delle chiavi effimere nel `client_metadata`, funzionamento del flag anti-replay, e gestione dei diversi schemi di prefisso;
- **Fase 3 (Response)**: corretta decifratura dei payload JWE, transizione atomica dello stato della sessione, validazione delle *disclosure* SD-JWT, verifica del Key Binding JWT (KB-JWT_{G}), e generazione del `responseCode` monouso;
- **Fase 4 (Fetch)**: corretta restituzione del VP Token_{G} al frontend, rifiuto di `responseCode` errati o già consumati, e cancellazione della sessione da Redis.

5.2.2 Testing manuale dei flussi di errore

Accanto ai flussi nominali, sono stati verificati manualmente i principali scenari di errore:

- invio di parametri mancanti o malformati (verifica della corretta restituzione di `400 Bad Request`);
- richiesta di sessioni scadute o inesistenti (verifica del `404 Not Found`);
- tentativo di consumare una sessione già in stato `response_received`;
- invocazione doppia del Request Object sulla stessa sessione (verifica del `403 Forbidden` per replay_{G});
- invio di un VP Token con nonce errato (verifica del `403 Forbidden` nella validazione del Key Binding).

5.3 Validazione tramite Conformance Test

5.3.1 La Conformance Test Suite della OpenID Foundation

La forma più autorevole di validazione per un'implementazione del protocollo OID4VP è il superamento dei Conformance Test ufficiali rilasciati dalla OpenID Foundation (OIDF_{G}). La suite di conformità, accessibile pubblicamente all'indirizzo `https://www.certification.openid.net/`, rappresenta lo strumento di riferimento adottato a livello internazionale per certificare la conformità delle implementazioni agli standard OpenID.

La suite opera come un simulatore di Digital Wallet: si sostituisce al wallet dell'utente e interagisce con il Verifier sotto test, verificando che le risposte, i formati, i parametri di sicurezza e i comportamenti di errore siano aderenti alle specifiche. L'obiettivo non è testare la logica interna del software, ma validare che il comportamento osservabile dall'esterno sia conforme allo standard.

5.3.2 Configurazione dell'ambiente di test

L'esecuzione dei Conformance Test ha richiesto una configurazione specifica dell'ambiente di lavoro. Poiché la suite OIDF opera da server remoti che devono poter raggiungere il Verifier sotto test, il backend locale è stato esposto su Internet tramite un tunnel ngrok:

```
MONOKEE_BASE_URL=https://dronish-unlabiate-jaylah.ngrok-free.dev
```

Questo URL pubblico è stato configurato come `VERIFIER_ENTITY_IDENTIFIER`, ovvero l'identità del Verifier nell'ecosistema OID4VP, e registrato nel test plan della suite OIDF con l'alias `monokee-oauth2-openid4vci`:

```
OPENID4VCI_AUTHORIZE_URL=https://www.certification.openid.net/test/a/monokee-
    oauth2-openid4vci/authorize
```

5.3.3 Categorie di test eseguiti

Il test plan OID4VP Verifier della OpenID Foundation comprende diverse categorie di verifiche, ciascuna progettata per validare un aspetto specifico della conformità del Verifier.

5.3.3.1 Test di flusso nominale

Questi test verificano che il Verifier sia in grado di completare con successo un'interazione completa con un wallet conforme:

- il Verifier genera correttamente un Request Object JWT con tutti i parametri obbligatori (`response_type`, `response_mode`, `nonce`, `state`, `dcql_query`, `client_id`);
- il Request Object è firmato con un algoritmo conforme e contiene header validi (`typ: oauth-authz-req+jwt`);
- il Verifier accetta un VP Token valido e restituisce il redirect URI con il `response_code`;
- il formato `dc+sd-jwt` è gestito correttamente (parsing, *disclosure*, firma dell'emittente).

5.3.3.2 Test negativi

Questi test verificano che il Verifier rifiuti correttamente input invalidi o malevoli:

- **Firma invalida sulla credenziale**: la suite invia un VP Token con una firma dell'emittente volutamente corrotta; il Verifier deve rifiutarla, dimostrando che la verifica crittografica è effettivamente operativa;
- **KB-JWT con nonce errato**: la suite invia un Key Binding JWT il cui nonce non corrisponde a quello generato nella fase di inizializzazione; il rifiuto dimostra la protezione contro gli attacchi di tipo *replay*;
- **KB-JWT con audience errata**: la suite invia un Key Binding JWT il cui `aud` non corrisponde all'identificativo del Verifier; il rifiuto impedisce che una presentazione generata per un altro Verifier venga accettata;
- **Integrità della sessione e anti-replay**: la suite tenta di riutilizzare un `request_uri` già consumato o di inviare una risposta su una sessione non più in stato `pending`; il Verifier deve rifiutare questi tentativi.

5.3.3.3 Verifica dei parametri di protocollo

La suite controlla la conformità formale dei parametri scambiati:

- la presenza e la correttezza del `Content-Type` nelle risposte;
- la coerenza tra `client_id` e lo schema di prefisso dichiarato;
- la struttura e i claim del Request Object JWT secondo la specifica JAR_{G};
- l'inclusione del `client_metadata` quando lo schema di prefisso lo richiede (`MUST_INCLUDE`).

5.3.4 Risultati e iterazioni correttive

L'esecuzione dei Conformance Test non è stata un'operazione puntuale con esito immediato, ma un processo iterativo. Le prime esecuzioni hanno evidenziato discrepanze tra l'implementazione iniziale e le aspettative della specifica, che hanno guidato cicli di correzione e ri-esecuzione.

Le principali aree su cui si è intervenuti includono:

- la corretta formattazione del `Content-Type` nelle risposte HTTP del Request Object;
- la gestione precisa del `response_mode` e delle sue varianti (`direct_post` e `direct_post.jwt`);
- la corretta inclusione dei parametri di cifratura nel `client_metadata` quando la negoziazione JWE è attiva;
- l'aderenza alla struttura DCQL_{G} nella serializzazione *snake_case* della query all'interno del Request Object;
- la gestione corretta degli errori provenienti dal wallet (campo `error` e `error_description`), con il redirect appropriato al frontend.

Questo ciclo di test, correzione e ri-esecuzione è stato fondamentale per affinare l'implementazione e portarla a un livello di conformità verificabile dall'esterno, costituendo il processo di validazione più significativo dell'intero progetto.

5.3.5 Valore della validazione tramite Conformance Test

Il superamento dei Conformance Test della OpenID Foundation fornisce un valore di validazione che va oltre il semplice testing funzionale:

- **Validazione dell'interoperabilità**: i test garantiscono che il Verifier sia in grado di interagire con qualsiasi Digital Wallet conforme allo standard OID4VP, indipendentemente dalla sua implementazione interna;
- **Validazione della sicurezza di protocollo**: i test negativi verificano che le contromisure di sicurezza funzionino correttamente, fornendo evidenza oggettiva della robustezza del sistema;
- **Validazione rispetto ai bisogni dell'utente**: il bisogno fondamentale per questo sistema è l'aderenza allo standard. Un Verifier OID4VP che non supera i Conformance Test è, per definizione, non conforme e quindi non utilizzabile in ecosistemi regolamentati, come quelli governativi europei per l'identità digitale dove la conformità agli standard EUDI è un requisito normativo.

Capitolo 6

Valutazione finale

Le attività di progettazione e sviluppo del progetto *monokee-relyingparty-core* si sono concluse con la realizzazione di un microservizio backend completo, in grado di ricoprire il ruolo di Relying Party$_{\text{G}}$ all'interno dell'ecosistema OID4VP$_{\text{G}}$.

6.1 Sintesi e bilancio generale

L'obiettivo primario del lavoro era costruire un sistema capace di richiedere, ricevere e validare credenziali digitali verificabili nel formato SD-JWT$_{\text{G}}$, garantendo al contempo l'interoperabilità con qualsiasi wallet conforme allo standard e la protezione crittografica dei dati in transito. Questo obiettivo è stato raggiunto nella sua interezza: il sistema supporta l'intero ciclo di vita delle *Verifiable Presentations* nelle sue quattro fasi operative (inizializzazione, generazione del Request Object, elaborazione della risposta e ritiro dei dati), ed è stato validato con successo attraverso i Conformance Test della OpenID Foundation.

Il valore aggiunto dell'implementazione risiede nelle sue proprietà architetturali, che la rendono adatta a un contesto di produzione *enterprise*:

- **Affidabilità e tolleranza ai guasti:** lo stato transazionale è interamente esternalizzato su Redis con TTL$_{\text{G}}$ automatici, garantendo che nessuna sessione rimanga orfana e che il sistema possa riprendersi da crash senza perdita di coerenza.
- **Sicurezza by design:** l'implementazione della Forward Secrecy$_{\text{G}}$ tramite chiavi effimere monouso con distruzione atomica, la verifica crittografica end-to-end delle Trust Chain$_{\text{G}}$ OpenID Federation, e la validazione multi-livello delle SD-JWT (firma issuer, hash *disclosure*, KB-JWT$_{\text{G}}$) costituiscono difese in profondità contro un ampio spettro di attacchi.
- **Multi-tenancy nativa:** l'isolamento logico per dominio, garantito sia a livello applicativo sia a livello di database (indici composti su MongoDB), rende il sistema immediatamente utilizzabile in contesti dove organizzazioni diverse condividono la medesima infrastruttura.
- **Scalabilità orizzontale:** non salvando dati nella memoria del singolo server, è possibile replicare facilmente il backend su più nodi dietro a un load balancer, senza dover modificare l'architettura del sistema.

Il risultato finale consiste, pertanto, in un componente *production-ready* progettato per essere integrato in un ecosistema di identità digitale reale, la cui conformità è stata certificata formalmente.

6.2 Validazione analitica dei risultati

6.2.1 Obiettivi raggiunti

Per fornire un'evidenza strutturata del livello di completamento del progetto, è utile incrociare sistematicamente gli obiettivi definiti in fase di progettazione con il loro stato finale di realizzazione. Nelle tabelle seguenti sono elencati i requisiti funzionali e non funzionali del sistema e il loro grado di soddisfacimento.

#	Requisito	Descrizione	Stato
RF-01	Inizializzazione sessione OID4VP	Il sistema deve creare una sessione transazionale, generare identificativi crittografici (nonce, requestId, sessionId, transactionId) e restituire un URL di autorizzazione.	Soddisfatto
RF-02	Generazione Request Object JWT	Il sistema deve costruire e firmare un JWT_{G} conforme a JAR_{G} (RFC 9101) contenente i parametri OID4VP, la query DCQL_{G} e le chiavi effimere.	Soddisfatto
RF-03	Elaborazione risposta del Wallet	Il sistema deve ricevere il VP Token_{G}, decifrare il payload JWE_{G} se necessario, validare la presentazione e generare un codice di ritiro monouso.	Soddisfatto
RF-04	Fetch e consegna della presentazione	Il sistema deve validare il responseCode, consegnare il VP Token verificato al Frontend e cancellare la sessione.	Soddisfatto
RF-05	Supporto a 6 schemi Client ID	Il sistema deve gestire i prefissi `redirect_uri`, `x509_san_dns`, `x509_hash`, `openid_federation`, `verifier_attestation`, `pre_registered`.	Soddisfatto
RF-06	Verifica crittografica SD-JWT	Il sistema deve parsare le SD-JWT, verificare la firma dell'issuer, validare gli hash delle disclosures e controllare il Key Binding JWT.	Soddisfatto
RF-07	Risoluzione Trust Chain (OpenID Fed.)	Il sistema deve risolvere ricorsivamente catene di fiducia tramite DFS_{G} con cycle detection e validare crittograficamente ogni anello della catena.	Soddisfatto
RF-08	Negoziazione algoritmica Verifier-Wallet	Il sistema deve negoziare dinamicamente gli algoritmi JWE, JWS_{G} e i formati VP supportati tra il Client configurato e le capability del wallet.	Soddisfatto
RF-09	Validazione DCQL completa	Il sistema deve verificare la corrispondenza delle credenziali presentate rispetto alla query DCQL, con supporto a `credentialSets` (valutati in), `claimSets`, path traversal e vincoli `vct`.	Soddisfatto
RF-10	Gestione chiavi effimere JWE	Il sistema deve generare coppie di chiavi asimmetriche effimere, salvarle in Redis con TTL, e distruggerle atomicamente dopo il primo utilizzo.	Soddisfatto
RF-11	CRUD_{G} Client via SCIM_{G} 2.0	Il sistema deve esporre endpoint REST_{G} per la creazione, lettura, aggiornamento e cancellazione delle configurazioni Client, conformi allo standard SCIM 2.0.	Soddisfatto
RF-12	Gestione errori del Wallet	Il sistema deve gestire le risposte di errore del wallet (`error`, `error_description`), aggiornare lo stato della sessione a `failed` e produrre un redirect URI con i parametri di errore.	Soddisfatto
RF-13	Anti-replay del Request Object	Il sistema deve impedire la generazione multipla del Request Object per la stessa sessione tramite il flag `isRequestUriConsumed`.	Soddisfatto
RF-14	Conversione camelCase ↔ snake_case	Il sistema deve convertire bidirezionalmente i formati interni e quelli di protocollo, preservando le chiavi speciali (`jwks`, `dc+sd-jwt`).	Soddisfatto
RF-15	Supporto JWT non firmati per pre_registered	Il sistema deve generare Unsecured JWT per i client pre-registrati, conformemente alla specifica OID4VP.	Soddisfatto

Tabella 6.1: Stato di completamento dei requisiti funzionali.

#	Requisito	Descrizione	Stato
RNF-01	Statelessness del backend	Nessuno stato transazionale in memoria locale; tutto esternalizzato su Redis.	Soddisfatto
RNF-02	Multitenancy$_G$	Isolamento dati per dominio a livello applicativo e di database.	Soddisfatto
RNF-03	Forward Secrecy	Chiavi effimere monouso con distruzione atomica post-uso.	Soddisfatto
RNF-04	Osservabilità	Logging strutturato con correlazione contestuale via AsyncLocalStorage.	Soddisfatto
RNF-05	Tolleranza ai guasti	TTL automatici su Redis per auto-cleanup delle sessioni scadute.	Soddisfatto
RNF-06	Conformità agli standard	Validazione tramite Conformance Test della OpenID Foundation.	Soddisfatto
RNF-07	Copertura di test automatizzati	Suite di test unitari e di integrazione per i componenti critici.	Non soddisfatto

Tabella 6.2: Stato di completamento dei requisiti non funzionali.

Il bilancio complessivo mostra che tutti i 15 requisiti funzionali e 6 dei 7 requisiti non funzionali sono stati soddisfatti. L'unico requisito non soddisfatto (RNF-07) riguarda la copertura di test automatizzati.

6.2.2 Debito tecnico e limiti

Oltre ai risultati positivi raggiunti, è fondamentale documentare con trasparenza il debito tecnico accumulato durante lo sviluppo. Analizzare queste limitazioni è essenziale per comprendere lo stato attuale del sistema e per pianificare gli interventi futuri. Di seguito vengono descritte le principali problematiche individuate, dalla più alla meno critica:

- **Assenza di test automatizzati (Criticità Alta):** l'assenza di un framework di testing dedicato è un problema rilevante per un sistema che gestisce identità digitali, in quanto l'attuale verifica del software dipende interamente da procedure manuali. Questa mancanza, dovuta ai tempi ristretti del progetto che hanno imposto di dare priorità alle funzionalità principali (*core*), aumenta il rischio che vulnerabilità o malfunzionamenti crittografici possano passare inosservati, con conseguenze potenzialmente gravi per la sicurezza.
- **Curva ellittica limitata a P-256 (Criticità Media):** l'`EphemeralKeyService` è attualmente implementato per generare chiavi effimere utilizzando esclusivamente la curva ellittica P-256, indipendentemente dall'algoritmo di cifratura (JWE) negoziato. Tale vincolo limita l'interoperabilità del sistema, in particolare con i *wallet* che richiedono curve crittografiche maggiormente robuste.
- ***Disclosure* di tipo array non supportate (Criticità Media):** il `SdJwtCredentialVerifierService` supporta unicamente le *disclosure* a tre elementi, non essendo in grado di gestire quelle a due elementi tipiche degli array. Questo limite impedisce la corretta verifica di credenziali SD-JWT che includono *claim* con valori multipli.
- **Assenza di *caching* per l'Entity Configuration (Criticità Media):** il `TrustAnchorService` effettua chiamate HTTP per ogni singola risoluzione della *Trust Chain*, senza avvalersi di alcun meccanismo di *cache*. Questo approccio genera latenza aggiuntiva e traffico di rete ridondante, riducendo l'efficienza complessiva.

6.2.3 Sviluppi futuri e miglioramenti proposti

Il debito tecnico precedentemente individuato, unito alla continua evoluzione degli standard di riferimento in materia di identità digitale, definisce in modo chiaro la direzione per i futuri sviluppi del progetto. Di seguito vengono descritti i principali miglioramenti proposti, organizzati in base alla loro priorità d'intervento:

- **Implementazione di una suite di test automatizzati:** per garantire la robustezza e l'affidabilità del sistema, è prioritario introdurre strumenti di testing moderni come Jest o Vitest. Questo permetterà di scrivere ed eseguire test unitari in modo automatizzato, con particolare attenzione ai servizi crittografici, simulando l'infrastruttura di appoggio.
- **Integrazione del supporto a curve ellittiche multiple:** per aumentare l'interoperabilità e il livello di sicurezza, il servizio `EphemeralKeyService` dovrà essere reso flessibile rispetto alla curva crittografica utilizzata. L'obiettivo è supportare standard diversificati e più robusti rispetto all'attuale P-256, estendendo la compatibilità a curve come P-384, P-521 e X25519.
- **Introduzione di un sistema di *caching* per le configurazioni:** al fine di ottimizzare le prestazioni e ridurre le chiamate di rete superflue, si propone di memorizzare in Redis i documenti di *Entity Configuration* e *Subordinate Statement*. Tali documenti verranno conservati in *cache* esclusivamente fino alla loro naturale scadenza, indicata esplicitamente dal *claim* `exp`.
- **Estensione del supporto SD-JWT agli array:** per consentire una gestione completa delle credenziali SD-JWT, sarà fondamentale completare l'implementazione relativa alle *disclosure* a due elementi. In questo modo il sistema sarà in grado di elaborare ed estrarre correttamente i *claim* che contengono liste multiple di valori.
- **Integrazione del supporto al formato mDoc_{G}:** guardando alle future applicazioni del sistema, un passo cruciale sarà lo sviluppo del servizio `MdocCredentialVerifierService`. Questo permetterà di gestire nativamente le credenziali basate sullo standard ISO 18013-5, un requisito indispensabile per supportare documenti di nuova generazione come la patente di guida digitale mobile.

6.3 Bilancio delle competenze

6.3.1 Competenze tecniche (Hard Skills)

Sviluppare un sistema di questa complessità ha richiesto di acquisire e consolidare diverse competenze tecniche, spingendosi ben oltre il tradizionale sviluppo web:

- **Crittografia applicata e protocolli di sicurezza:** applicazione concreta della firma digitale asimmetrica (JWS), della cifratura ibrida (JWE), della generazione di chiavi effimere per la Forward Secrecy, e della verifica di catene di fiducia. L'implementazione del parser SD-JWT ha rappresentato la sfida più complessa, richiedendo la convalida di hash, la ricostruzione di digest crittografici e l'analisi di Key Binding JWT.
- **Protocolli di identità decentralizzata:** studio approfondito dell'ecosistema normativo (OID4VP, DCQL, SD-JWT, OpenID Federation 1.0, JAR). Leggere, interpretare e tradurre in codice funzionante un *draft* di specifica internazionale rappresenta una competenza fondamentale in questo settore.
- **TypeScript e Node.js in ambito server-side:** consolidamento di TypeScript in scenari critici, sfruttando la tipizzazione statica per modellare i contratti tra servizi, e padronanza del runtime asincrono Node.js.
- **Architettura a microservizi e pattern:** progettazione modulare consapevole, sfruttando pattern come *Layered Architecture*, *Facade*, *Pipeline* e *Strategy*, bilanciando costi e benefici delle scelte architetturali.

- **Gestione di database dual-layer:** utilizzo combinato di Redis per lo stato transitorio e MongoDB per la persistenza configurativa, sfruttando funzionalità avanzate per rispondere a rigorosi requisiti di sicurezza.

6.3.2 Competenze trasversali (Soft Skills)

L'attività progettuale ha favorito inoltre lo sviluppo di importanti competenze trasversali quali:

- **Problem solving su standard in evoluzione:** gestire l'incertezza delle specifiche tecniche ancora in bozza, consultando le documentazioni ufficiali per prendere decisioni motivate.
- **Autonomia progettuale e responsabilità:** saper valutare i pro e i contro delle scelte tecniche, documentandone i motivi per facilitare la manutenzione futura del progetto.
- **Pensiero autocritico:** valutare in modo oggettivo il codice scritto, riconoscendone i difetti (debito tecnico) e la loro gravità per pianificare i miglioramenti futuri.
- **Comunicazione tecnica:** l'abilità di spiegare in modo chiaro concetti e processi crittografici complessi, traducendoli in documentazione e codice comprensibili per gli altri sviluppatori.

6.4 Riflessione e chiusura

L'esperienza di tirocinio ha rappresentato un'opportunità formativa preziosa, consentendo di consolidare le nozioni teoriche apprese durante il percorso accademico e di applicarle all'interno di un contesto aziendale reale. La progettazione e lo sviluppo del modulo Relying Party hanno richiesto un impegno costante, rendendo necessaria la gestione dell'intero ciclo di vita del prodotto software: dall'analisi dei requisiti fino alla fase di collaudo. A questo processo si è aggiunta la complessità del dominio applicativo. Se, infatti, a livello teorico una firma digitale può apparire come un'operazione elementare, nella realtà ingegneristica essa impone la negoziazione dinamica degli algoritmi, la validazione rigorosa dei certificati X.509 nei JWK_{G} e la prevenzione attiva delle vulnerabilità crittografiche.

Inoltre, l'ingegneria del software applicata a contesti critici, quale l'identità digitale decentralizzata, richiede una profonda assunzione di responsabilità.

Infine, l'inserimento nel team di Monokee si è rivelato altamente stimolante: la collaborazione continua con il tutor aziendale e con i colleghi ha arricchito il percorso non solo dal punto di vista tecnico, ma anche metodologico, evidenziando il ruolo centrale della comunicazione e della pianificazione nello sviluppo software. I traguardi raggiunti confermano la validità del lavoro svolto, ponendo solide basi per il futuro percorso professionale.

Bibliografia

Siti web consultati

17, ISO/IEC JTC 1/SC. *ISO/IEC 18013-5:2021 - Personal identification — ISO-compliant driving licence*. 2021. URL: https://www.iso.org/standard/69084.html.

Consiglio dell'Unione Europea, Parlamento Europeo e. *Regolamento (UE) 2024/1183 (eIDAS 2.0)*. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/1183/oj.

Expert Group, eIDAS. *European Digital Identity Wallet Architecture and Reference Framework (ARF)*. 2023. URL: https://github.com/eu-digital-identity-wallet/eudi-doc-architecture-and-reference-framework.

Fett, D., B. Campbell e K. Yasuda. *Selective Disclosure for JWTs (SD-JWT)*. IETF Internet-Draft. URL: https://datatracker.ietf.org/doc/draft-ietf-oauth-selective-disclosure-jwt/.

Foundation, OpenID. *OpenID Federation 1.0*. URL: https://openid.net/specs/openid-federation-1_0.html.

— *OpenID for Verifiable Credential Issuance (OID4VCI)*. URL: https://openid.net/specs/openid-4-verifiable-credential-issuance-1_0.html.

— *OpenID for Verifiable Presentations (OID4VP)*. URL: https://openid.net/specs/openid-4-verifiable-presentations-1_0.html.

Hunt, P. et al. *RFC 7643: System for Cross-domain Identity Management (SCIM): Core Schema*. 2015. URL: https://datatracker.ietf.org/doc/html/rfc7643.

Jones, M., J. Bradley e N. Sakimura. *RFC 7515: JSON Web Signature (JWS)*. 2015. URL: https://datatracker.ietf.org/doc/html/rfc7515.

— *RFC 7519: JSON Web Token (JWT)*. 2015. URL: https://datatracker.ietf.org/doc/html/rfc7519.

Jones, M. e J. Hildebrand. *RFC 7516: JSON Web Encryption (JWE)*. 2015. URL: https://datatracker.ietf.org/doc/html/rfc7516.

Microsoft. *TypeScript: Typed JavaScript at Any Scale*. URL: https://www.typescriptlang.org/.

MongoDB: The Developer Data Platform. URL: https://www.mongodb.com/.

Monokee: Identity and Access Management. URL: https://www.monokee.com/.

Node.js: JavaScript runtime built on Chrome's V8 JavaScript engine. URL: https://nodejs.org/.

Redis: The open source, in-memory data store. URL: https://redis.io/.

Skokan, Filip. *jose: JSON Web Almost Everything*. URL: https://github.com/panva/jose.

W3C. *Verifiable Credentials Data Model v2.0*. URL: https://www.w3.org/TR/vc-data-model-2.0/.